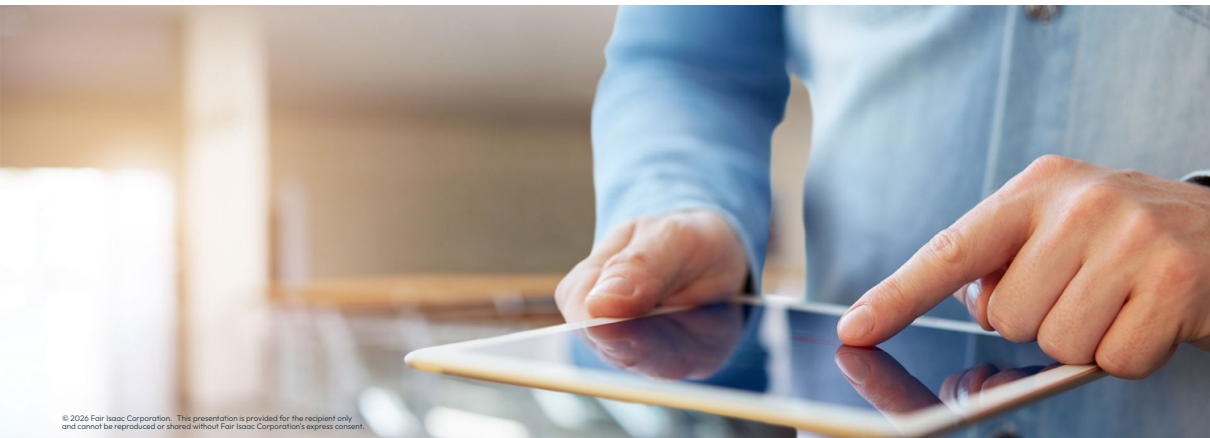




Introduction to working with FICO® Xpress Mosel

FICO Xpress Training





Course topics

Overview

- Introduction, Xpress overview
- Modeling with Mosel
- Embedding Mosel models
- Summary and further information



Introduction, Xpress overview

- Overview of Xpress
- Xpress Workbench demonstration
- VS Code Mosel extension



Topics

- Introduction to Xpress
- Modeling with Mosel:
 - Linear and Mixed Integer Programming (LP and MIP)
 - Accessing data sources
 - Programming language features
- Embedding models in applications

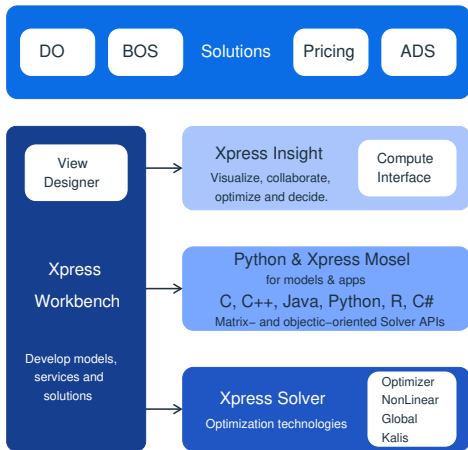


Overview of Xpress

- Xpress innovations
- Components of FICO Xpress

FICO Xpress Optimization

The most complete optimization software on the market



Xpress Solutions

- Point-and-click applications designed for business users, includes **Decision Optimizer** and **Business Outcome Simulation**.

Xpress Technology

- **Xpress Insight** – Rapidly deploy analytic & optimization models as powerful applications.
 - **Compute Interface** for optimization execution.
- **Python or Xpress Mosel** – develop models and apps in the preferred language.
- **Xpress Solver** – Optimization algorithms and technologies to solve linear, mixed integer, non-linear, global and constraint programming problems.
- **Xpress Workbench** – IDE for developing optimization models, services, and solutions.
 - Drag & Drop view configuration using **View Designer**

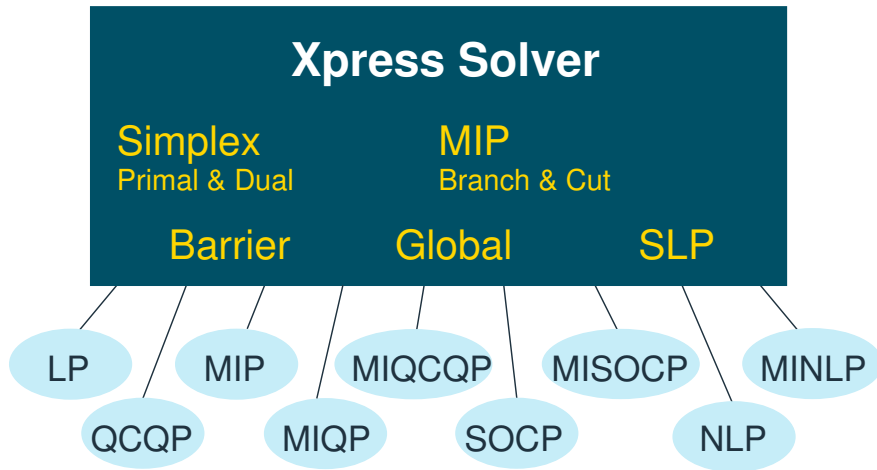
Overview of Xpress

- Optimization algorithms
 - solve different classes of problems
 - built for speed, robustness and scalability

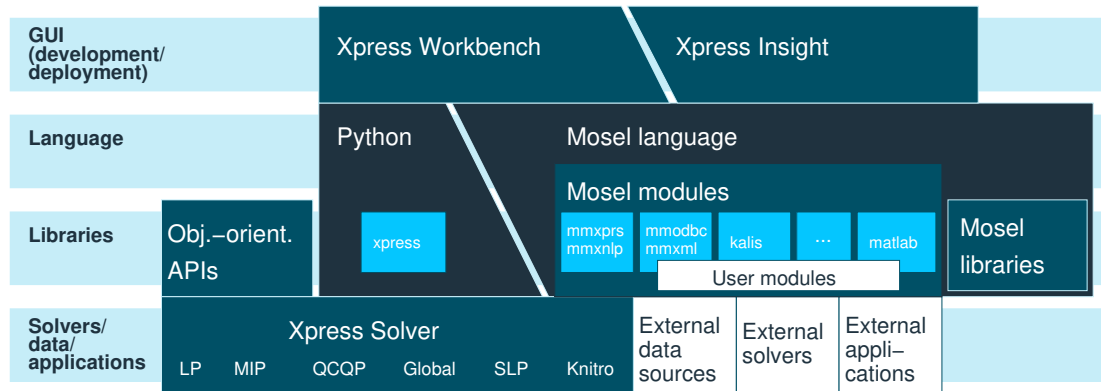
Overview of Xpress

- Optimization algorithms
 - solve different classes of problems
 - built for speed, robustness and scalability
- Modeling interfaces
 - Mosel
 - formulate model and develop optimization methods using Mosel language / environment
 - Python and object-oriented APIs
 - build up models in your application code using APIs for C, C++, Java, .NET, Python, R, Julia, MATLAB
- Application development
 - Xpress Insight
 - deploy multi-user optimization applications

Optimization algorithms



Overview of Xpress



43 years of business transformation with FICO Xpress Optimization

- Solving

1983: LP solver running on PCs

1992: parallel MIP (1997 on distributed PC/Linux networks)

1995/1996: commercial branch and cut algorithm

1998: bound switching in dual simplex

2003: lift-and-project cuts

2009: parallel MIP heuristics

2010: LP/MIP solver crosses 64-bit coefficient indexing threshold

2013: automatic solver selection for NLP

2014: parallel simplex

2016: task-based parallel MIP

2017: Barrier warm start and parallel crossover; parallel black box optimization

2018: LP folding

2019: iterative solution refiner

2020: Machine Learning for scaling; remote execution

2021: ML for cutting in tree search

2023: Global optimization solver

2025: PDHG; LP solving on GPU

43 years of business transformation with FICO Xpress Optimization

- Modeling

1983: general purpose algebraic modeling language (mp-model)

2001: algebraic modeling language combining modeling, solving and programming (Mosel)

2005: profiler and debugger for a modeling language

2005: user-controlled parallelism at the model level

2010: algebraic modeling language supporting distributed computing

2012: remote invocation without local installation (XPRD)

2014: parallel profiler and debugger; robust optimization; encryption

2015: model annotations and mosel doc

2016: native language support (XPRNLS), Unicode; remote invocation protocol

2017: cloud-based model and solution development

2018: Mosel free and extensible with new solvers

2019: dynamic package loading; namespaces; hashmap array type

2020: cloning of models and data sharing; constants of native types

2022: union types; reflection

43 years of business transformation with FICO Xpress Optimization

- Business user enablement

2012: Xpress Insight first released

2015: VDL for GUI definition

2016: Xpress Workbench for end-to-end optimization app development

2017-18: Pre-configured applications for Machine Learning (Pattern Discovery), ML-driven Forecasting

2019: View Designer for drag & drop UI configuration

2020: Compute Interface; Python-Insight app development

2022: Custom Progress Reporting

Our experience

Supply chain optimization

Portfolio generation + maintenance

Personnel scheduling

Production scheduling

Process industries (blending)

Vehicle routing

Marketing optimization

Generalized assignment (e.g. facility location)

Auctions

Airline operations

Electricity generation, supply and pricing

Mosel

- A modeling and solving environment
 - integration of modeling and solving
 - programming facilities
 - open, modular architecture
- Interfaces to external data sources (*e.g.* ODBC, host application) provided
- Language is concise, user friendly, high level
- Best choice for rapid development and deployment

Mosel: Components and interfaces

- **Mosel Native Interface** (NI): to provide new or extend existing functionality of the Mosel language

⇒ *module (DSO)*

Modules of the Mosel distribution:

- *solvers*: mmxprs, mmquad, mmxnlp, kalis, nlsolv
- *data handling*: mmodbc, mmoci, mmsheet, mmxml
- *model handling, utilities*: mmjobs, mmsystem, mmhttp, mmssl
- *graphics, GUI*: mmsvg, mminsight

Mosel: Components and interfaces

- **Mosel Native Interface** (NI): to provide new or extend existing functionality of the Mosel language

⇒ *module (DSO)*

Modules of the Mosel distribution:

- *solvers*: mmxprs, mmquad, mmxnlp, kalis, nlsolv
- *data handling*: mmodbc, mmoci, mmsheet, mmxml
- *model handling, utilities*: mmjobs, mmsystem, mmhttp, mmssl
- *graphics, GUI*: mmsvg, mminsight
- **Xpress Workbench**: graphical user interface
 - model editor, debugging, deployment to Xpress Insight

Mosel: Components and interfaces

- **Mosel Native Interface** (NI): to provide new or extend existing functionality of the Mosel language

⇒ *module (DSO)*

Modules of the Mosel distribution:

- *solvers*: mmxprs, mmquad, mmxnlp, kalis, nlsolv
- *data handling*: mmodbc, mmoci, mmsheet, mmxml
- *model handling, utilities*: mmjobs, mmsystem, mmhttp, mmssl
- *graphics, GUI*: mmsvg, mminsight
- **Xpress Workbench**: graphical user interface
 - model editor, debugging, deployment to Xpress Insight
- **Visual Studio Code**: alternative editor with Mosel language support
 - syntax highlighting, run and debug Mosel models via the Xpress VS Code extension

Mosel: Components and interfaces

- **Mosel Native Interface** (NI): to provide new or extend existing functionality of the Mosel language

⇒ *module (DSO)*

Modules of the Mosel distribution:

- *solvers*: mmxprs, mmquad, mmxnlp, kalis, nlsolv
- *data handling*: mmodbc, mmoci, mmsheet, mmxml
- *model handling, utilities*: mmjobs, mmsystem, mmhttp, mmssl
- *graphics, GUI*: mmsvg, mminsight
- **Xpress Workbench**: graphical user interface
 - model editor, debugging, deployment to Xpress Insight
- **Visual Studio Code**: alternative editor with Mosel language support
 - syntax highlighting, run and debug Mosel models via the Xpress VS Code extension
- **Tools**: debugger, profiler, *mosel/doc*

FICO Xpress Mosel extension for Visual Studio Code

- Install from the **VS Code Marketplace**: search **FICO Xpress Mosel (BETA)**
- Activates automatically when a `.mos` file is open
- Language features: syntax highlighting, code folding, jump to definition, code completion, inline help, real-time diagnostics

```
13  model Prime
27
28  n:= 2
29  repeat
30  | while (not (n in SNumbers)) n+=1
31  | SPrime+={ n } ! n is a prime number
32
33  i:= n
34  while (i<=LIMIT) do ! Remove n and all its multiples
35  | SNumbers-={ i }
36  | i+=n
37  end-do
38  until SNumbers = { }
```

Expression: `n > 20`

Mosel and Optimizer Consoles

- Stand-alone command line executables with text interfaces
- Useful for simple deployment using batch/script files
- Available for all platforms supported by Xpress



Xpress Workbench demonstration

Xpress Workbench demonstration

- A tool for creating, editing, and debugging Xpress Insight apps (implemented with Mosel or Python) and Xpress Mosel models
- Insight Workbench enables you to do the following tasks:
 - create and edit source files (Mosel models, Python programs and VDL views)
 - publish Xpress Insight apps to an Insight server
 - debug an Insight scenario while it is executing within an Xpress Insight server
 - build an Xpress Insight app archive
 - debug a Mosel model in a standalone instance of Mosel
 - compile a Mosel model into a BIM file

Creating an Xpress Mosel project

FICO® Xpress Workbench

FICO.comCommunityHELP ▾

Recent projects

Create project

Open existing file or folder

Examples

- Introductory examples
- Insight examples
- Nonlinear examples
- Calling R from Mosel
- Calling Python from Mosel
- Browse all examples

Xpress Documentation

Ask the Xpress community

Choose from the options below. You will be asked to select a folder location in which the project files will be created.

Create an Xpress Mosel project

The project will contain a simple Mosel source file template. Choose this option if you want to develop a new Mosel model which is not intended to run within Xpress Insight.

CREATE MOSEL PROJECT

Create an Xpress Insight (Mosel) project

The project will contain templates for a Mosel model, custom views and companion file. Choose this option if you want to develop a new Xpress Insight app in Mosel.

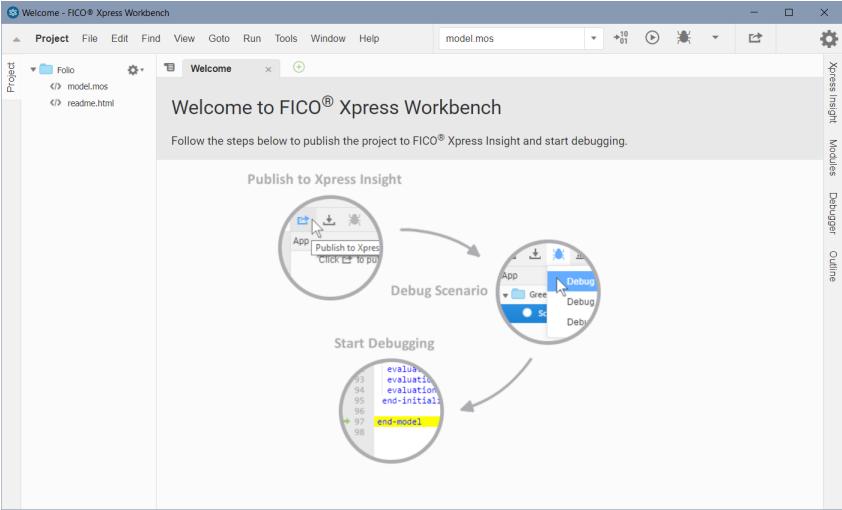
CREATE INSIGHT (MOSEL) PROJECT

Create an Xpress Insight (Python) project

The project will contain templates for a Python model, custom views and companion file. Choose this option if you want to develop a new Xpress Insight app in Python.

CREATE INSIGHT (PYTHON) PROJECT

Exploring the Workbench workspace



Xpress Workbench demonstration

- Workbench panes
 - model editor (central window)
 - project directory navigation and command history (left)
 - model output and execution log information (bottom)
 - debugging, deployment and collaboration information (right)
 -  Workspace preferences (settings).
 -  Toggle full-screen view for logging pane.

Xpress Workbench demonstration

- Editor

- code folding and breakpoint markers appear in the grey area immediately left to the text



Open a new file/tab



Subdivide and re-arrange panes in the editor window



Code folding for blocks of Mosel statements



Unfold folded code



Line position markers during debugging

Xpress Workbench demonstration

- Model execution
 - the name of the model is selected in the box next to these buttons, it may be different from the model(s) opened in the editor
 -  Compile a model.
 -  Execute (compile/load/run) a model.
 -  Execute a model in debug mode.
 -  Open Compiler Options or Run Dialog.
 - alternatively, use menu *Run* to compile or run a model

Debugging an Xpress Mosel project

The screenshot displays the FICO Xpress Workbench interface for debugging a Mosel project named 'foliopl.mos'.

Main Editor (foliopl.mos):

```
32 sum(s in RISK) frac(s) <= 1/3
33
34 ! Minimum amount of North-American values
35 sum(s in NA) frac(s) >= 0.5
36
37 ! Spend all the capital
38 sum(s in SHARES) frac(s) = 1
39
40 ! Upper bounds on the investment per share
41 forall(s in SHARES) frac(s) <= 0.3
42
43 ! Solve the problem
44 maximize(Return)
45
46 ! Solution printing
47 if getprobatat=XPRS_OPT then
48   writeln("Total return: ", getobjval)
49   forall(s in SHARES) writeln(s, " ", getsol(frac(s))*100, "%")
50 end-if
51
52 end-model
```

Command Window (foliopl.mos - Ru):

```
Compiling foliopl.mos to out\foliopl.bim with -G
Running model
Debug session 98f11d306a1d75c7fb52eeb418456ae2 started
4: 0%
5: 6.66666667%
6: 30%
7: 0%
8: 0%
9: 13.33333333%
10: 0%
The model is suspended after execution
```

Debugger Panels:

- Watch Expressions:** A table with columns 'Expression', 'Value', and 'Type'. It includes a text input field 'Type your expression here...'.
- Call Stack:** A table with columns 'Function', 'File', 'Ln', and 'Col'. It shows '(Model ended)' at 'foliopl.mos' line 52, column 2.
- Variables:** A table with columns 'Variable', 'Value', and 'Type'. It lists 'Globals' and 'entry' variables.
- Outline:** A tree view showing the project structure, including 'Folio', 'out', and 'foliopl.mos'.

Xpress Workbench demonstration

- Debugger
 - breakpoints are set by clicking onto the gray area (left to the line number if it is displayed) preceding each row in the editor window
 - breakpoint conditions can be added via the right mouse button menu on the breakpoint icon
 -  Delete breakpoint/desactivated breakpoint.
 -  Delete a conditional/desactivated conditional breakpoint.

Xpress Workbench demonstration

- Navigating in the debugger
 - buttons are activated during debug runs
 -  Activate/deactivate all breakpoints.
 -  Start/stop the debugger.
 -  Resume/suspend model execution.
 -  Step over an expression.
 -  Step into an expression.
 -  Step out of an expression.
 -  Don't pause on exceptions.

Xpress Workbench demonstration

- Deployment to Xpress Insight
 - only if Xpress Insight has been installed
 -  Publish selected model to Insight.
 -  Build an Insight app archive.
 -  Debug a scenario.
 -  Edit Tableau workbooks.
 -  Refresh Insight scenario tree.

Project work [P-1]: Working with Workbench

- Start up Xpress Workbench
 - double click on a Mosel model, or
 - double click on the Workbench icon  on your desktop
- Execute a model and observe the changes to the displays in Workbench.
- Search for information on 'exportprob' and 'verbose' in the online help.

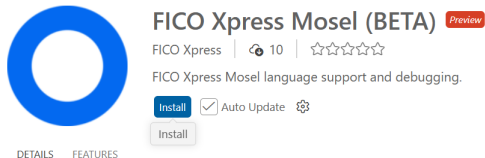
Reference material

- The manual [Getting Started with Xpress](#) introduces first time or occasional users to modeling with Mosel and BCL, or the direct Optimizer interface
- The [Evaluators Guide](#) and [Advanced Evaluators Guide](#) provide a quick walk-through of the Getting Started examples and some more advanced features
- The [Xpress Workbench Guide](#) explains how to edit models with this GUI

 VS Code Mosel extension

VS Code Mosel extension

- An extension for Visual Studio Code that brings full Mosel language support inside the editor
- Install from the VS Code Marketplace (Ctrl+Shift+X): search for **FICO Xpress Mosel (BETA)**
 - activates automatically when a .mos file is open
 - language features work without Xpress; runtime features require a valid Xpress installation
- Installing the Mosel extension



Syntax highlighting, code folding, and navigation

- **Syntax highlighting**: keywords, string literals, numeric constants, comments, and identifiers each receive distinct visual treatment

```
model Prime
  parameters
    LIMIT=100 .....! Search for prime numbers in 2..LIMIT
  end-parameters

  declarations
    SNumbers: set of integer .....! Set of numbers to be checked
    SPrime: set of integer .....! Set of prime numbers
  end-declarations

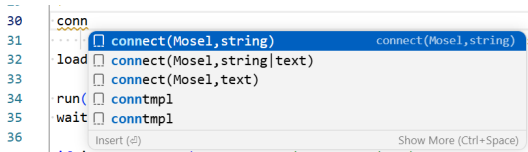
  SNumbers:= {2..LIMIT}

  writeln("Prime numbers between 2 and ", LIMIT, ":")
```

- **Code folding**: collapse and expand blocks such as parameters, declarations, initializations, procedure, and function
 - click the arrow in the editor gutter
- **Jump to definition**: press F12 on any symbol to jump to its declaration
 - works across files linked via uses, import, or include

Code intelligence

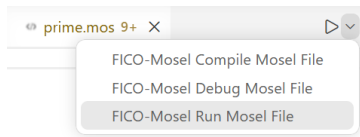
- **Code completion:** suggestions from the [Mosel Language Reference](#) and local declarations, with full signatures and documentation in the [detail panel](#)



- **Inline help:** hover over any symbol to see its declaration and documentation without leaving the current file
- **Real-time diagnostics:** errors and warnings appear inline from the Mosel parser, without triggering a full compile

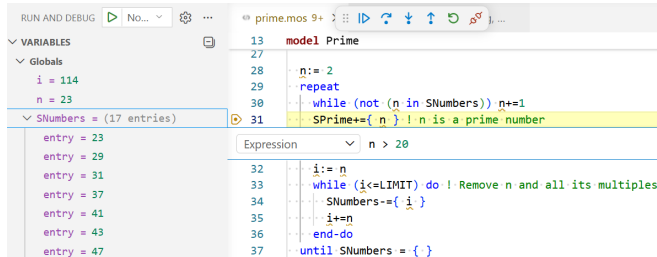
Runtime features

- A **play button** in the editor title bar opens a quick menu to compile, run, or debug the current `.mos` file
 - all commands are also available in the command palette (`Ctrl+Shift+P`)
- **Compile**: compile Mosel source files into portable binary (`.bim`) format
- **Run**: execute Mosel models with live output in a VS Code terminal
- **Debug**: full debugging support with breakpoints, variable inspection, and expression evaluation
- Runtime parameters can be passed via `.vscode/tasks.json` to automate compile and debug workflows



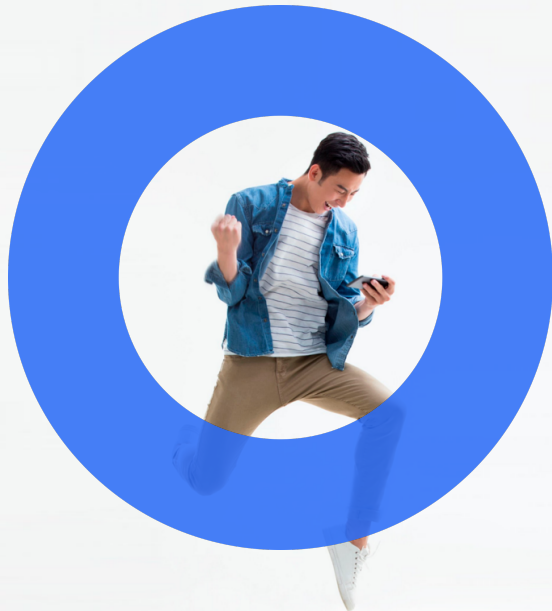
Debugging a Mosel model

- **Breakpoints**: set anywhere in Mosel code, including conditional breakpoints that trigger only when specific conditions are met
- **Variable inspection**: inspect variables in the Variables pane and the **Variable Inspector** panel, which expands sets and arrays into individual elements with their indices
- **Debug Console**: evaluate Mosel expressions and built-in functions against live model state, without modifying source code



Reference material

- The [Getting Started with Xpress](#) manual and the [Evaluators Guide](#) provide a quick introduction to modeling with Mosel
- The [Xpress Mosel Reference Manual](#) documents all language constructs and built-in functions supported by the extension
- The [FICO Xpress Mosel extension](#) is available on the VS Code Marketplace





Modeling with Mosel

- Modeling basics
- Accessing data sources
- Advanced modeling topics
- Programming language features
- Mosel modules and packages





Modeling basics (LP)

- A first model
- Data structures and loops
- Model building style

Topics

- Definition of decision variables, constraints and objective
- Solving with Xpress Optimizer
- Solution output

Example: Chess problem

- A joinery makes two different sizes of boxwood chess sets.
- The small set requires 3 hours of machining on a lathe, and the large set requires 2 hours. There are 4 lathes with skilled operators who each work a 40 hour week.
- The small chess set requires 1 kg of boxwood, and the large set requires 3 kg. Only 200 kg of boxwood can be obtained per week.

Example: Chess problem

- Each of the large chess sets yields a profit of \$20, and one of the small chess sets has a profit of \$5.
- How many sets of each kind should be made each week so as to maximize profit?

Chess problem:

Mathematical formulation

- To model a problem mathematically we need **three key components**:
 - **Decision variables**: what we want to determine (quantities to produce)
 - **Constraints**: limitations and requirements (resource availability, capacity)
 - **Objective function**: what we want to optimize, a criterion to evaluate solution quality (maximize profit)

- Formulating the chess problem:

x_l – quantity of large chess sets made

x_s – quantity of small chess sets made

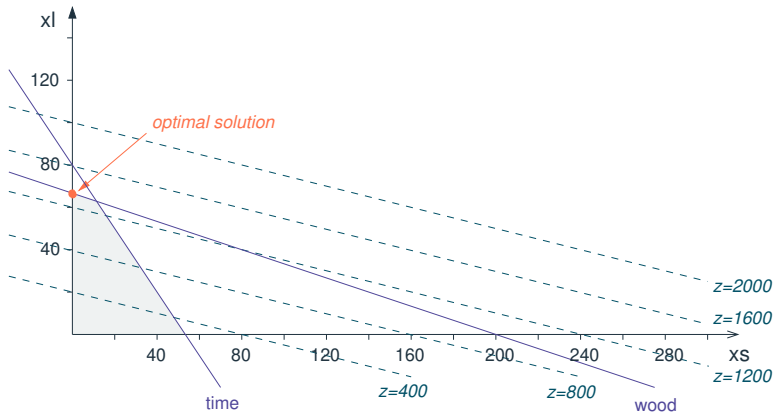
$$\max \quad z = 5 \cdot x_s + 20 \cdot x_l$$

$$\text{s.t.} \quad 3 \cdot x_s + 2 \cdot x_l \leq 160 (= 4 \cdot 40) \quad (\text{lathe time})$$

$$x_s + 3 \cdot x_l \leq 200 \quad (\text{wood})$$

$$x_s, x_l \geq 0$$

Chess problem: Graphical solution



Chess problem:

Model Chess 1

```
model "Chess 1"
  uses "mmxprs"                                ! Use Xpress Optimizer for solving

  declarations
    xs: mpvar                                  ! Number of small chess sets
    xl: mpvar                                  ! Number of large chess sets
  end-declarations

  3*xs + 2*xl <= 160                            ! Constraint: limit on working hours
  xs + 3*xl <= 200                             ! Constraint: raw mat. availability

  maximize(5*xs + 20*xl)                       ! Objective: maximize total profit

end-model
```

Starting and ending a Mosel model

```
model "Chess 1"  
...  
end-model
```

Preamble

- uses statement: Say we will use the Xpress Optimizer library, so that we can solve our problem
- Options:
 - noimplicit: force all objects to be declared
 - explterm: Use ';' to mark line ends

```
uses 'mmxprs'  
options noimplicit  
options explterm
```

Decision variables

- `mpvar` means **mathematical programming variable** or **decision variable**
- Decision variables are unknowns: they have no value until the model is run, and the optimizer finds values for the decision variables

Decision variables

- In optimization problems, decision variables are often just called **variables**
- In computer programs, a variable can be used to refer to many different types of objects
- For instance, in Mosel models, a **program variable** can be used to refer to a **decision variable**, as well as integers, reals, *etc.*

Bounds on decision variables

- Variables can take values between 0 and infinity by default
- Other bounds may be specified

```
x <= 10  
y(1) = 25.5  
y(2) is_free  
z(2,3) >= -50  
z(2,3) <= 50
```

Constraints

- Have type `linctr` – linear constraint

```
declarations
  Wood: linctr
  Inven: array(1..10) of linctr
end-declarations
```


Constraints

- The 'value' of a constraint entity is a linear expression of decision variables, a constraint type, and a constant term
- Set using an assignment statement
- Constraints can be named (or anonymous)

Wood := xs + 3*x1 <= 200

Objective function

- An objective function is just a constraint with no constraint type

```
declarations
```

```
    MinCost: lincstr
```

```
end-declarations
```

```
MinCost := 10*x(1) + 20*x(2) + 30*x(3) + 40*x(4)
```

Optimization & matrix generation

- Generate the matrix and solve the problem:

```
minimize(MinCost)
maximize(5*xs + 20*xl)
```

- Explicit matrix loading:

- performed automatically by `minimize / maximize`

```
loadprob(MinCost)
```

- Matrix export:

- problem definition in Mosel:

```
exportprob(0, "explout", MinCost)      ! 0: Output in LP format (human readable)
```

- problem loaded into the Optimizer:
 - preferable for solver tuning or debugging

```
loadprob(MinCost)
writeprob("explout.mps", "")           ! Output as MPS file
```

Viewing the solution

- Can access and manipulate the solution values within the model

```
writeln('Solution: ', getobjval)
```

```
writeln('xs = ', getsol(xs))
```

```
writeln('xl = ', getsol(xl))
```

```
write('Wood: ', getact(Wood), ' ')
```

```
writeln(getslack(Wood))
```

- Solution values of constraints
activity value + slack value = RHS

Viewing the solution

- Use case to handle all problem states

```
case getprobat of
  XPRS_OPT: writeln('optimal')
  XPRS_INF: writeln('infeasible')
  XPRS_UNB: writeln('unbounded')
  XPRS_UNF: writeln('unfinished')
else
  writeln('unexpected problem status!')
end-case
```

Project work [C-1]: Chess problem

- Execute the model `chess1.mos`.
- Add printing of the solution values.
- Is the solution realistic/desirable?
- Constrain the variables to take integer values only.
- Add output of constraint activity and slack values.

Project work [C-1]:

Chess problem

- Executing model `chess1.mos` with Xpress Workbench:
 - double click on the model file to start Workbench or open the file from within the GUI
 - click on the run button: 
- Model execution from the command line:

```
mosel exec chess1.mos
```

- or shorter:

```
mosel chess1
```

Solution:

Completed model Chess 1

```
model "Chess 1 (completed)"
  uses "mmxprs"                                ! Use Xpress Optimizer for solving

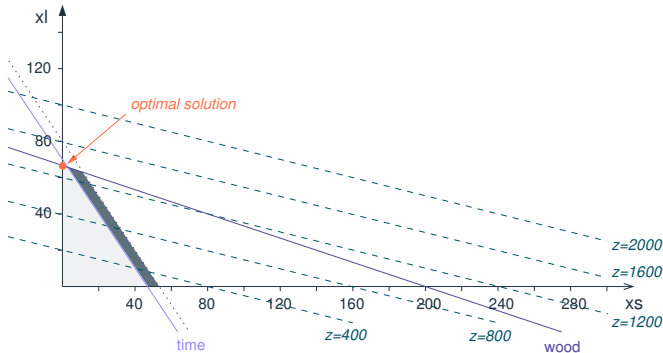
  declarations
    xs,xl: mpvar                                ! Decision variables
  end-declarations

  Profit:= 5*xs + 20*xl                        ! Name the objective function
  Time:=   3*xs + 2*xl <= 160                  ! and the constraints
  Wood:=   xs + 3*xl <= 200
  xs is_integer; xl is_integer                ! Integrality constraints

  maximize(Profit)                             ! Objective: maximize total profit
  if getprobat=XPRS_OPT then
    writeln("Solution: ", getobjval)           ! Print objective function value
    writeln("xs: ", getsol(xs), " xl: ", getsol(xl)) ! Print solution values
    write("Time: ", getact(Time), " ", getslack(Time)) ! Constraint activity and slack
  end-if
end-model
```


Solution analysis

- What happens if machines operate 35 instead of 40 hours?



- Calculate spare capacity: `getslack`, `getactivity`
 - $\text{constraint activity} + \text{slack} = \text{RHS}$

Data structures

- **Set:** unordered collection of objects of the same type
 - used as **index sets**
 - special type **range sets** (= interval of integers)
- **Array:** multidimensional table of objects of the same type
 - used for data, decision variables, constraints
 - may be **sparse** or **dense**

Arrays and loops:

Model Chess 3

```
uses "mmxprs"

declarations
  R = 1..2                                ! Index range
  DUR, WOOD, PROFIT: array(R) of real    ! Coefficients
  x: array(R) of mpvar                  ! Array of variables
end-declarations

DUR      :: [3, 2]                       ! Initialize data arrays
WOOD     :: [1, 3]
PROFIT   :: [5, 20]

sum(i in R) DUR(i)*x(i) <= 160           ! Constraint definition
sum(i in R) WOOD(i)*x(i) <= 200
forall(i in R) x(i) is_integer
maximize(sum(i in R) PROFIT(i)*x(i))
writeln("Solution: ", getobjval)
```

Data declaration

```
declarations
  N WEEKS = 20                                ! Integer constant
  DATA_DIR = 'c:/data'                       ! String constant
  NPROD: integer                             ! Integer variable
  SCOST: real                                 ! Real variable
  DIR: string                                ! String variable
  IF_DEBUG: boolean                          ! Boolean variable

  PRODUCTS = {"P1", "P2", "P4"}              ! Constant set of string
  S: set of integer                          ! Variable set of integer
  R: range                                    ! Range of integers
  COST: array(1..3,1..4) of real             ! Array of real
end-declarations
```

Data initialization

```
NPROD:= 50  
SCOST:= 5.4  
DIR:= 'c:/data'  
IF_DEBUG:= true  
  
S:= {10, 0, -5, 13}  
R:= 1..NPROD  
COST:: [11, 12, 13, 14,  
        21, 22, 23, 24,  
        31, 32, 33, 34]
```

Summations

- Sum up an array of variables in a constraint

```
Ctrl1:= sum(p in 1..10) (RES(p)*buy(p) + sell(p)) <= 100
```

```
Ctrl2:= sum(p in PRODUCTS) (buy(p) + sum(r in 1..5) make(p,r)) <= 100
```

```
Ctrl3:= sum(p in 1..NP) (2*CAP(p)*buy(p)/10 +  
    SCAP(p)*sell(p)) <= MAXCAP
```

Loops

- Use a loop to assign an array of constraints

```
forall(t in 2..NT)  
    Inven(t) := bal(t) = bal(t-1) + buy(t) - sell(t)
```

Loops

- Use do/end-do to group several statements into one loop

```
forall(t in 1..NT) do
  MaxRef(t) := sum(i in PRODUCTS)
    use(i,t) <= MAXREF(t)

  Inven(t) := store(t) = store(t-1) + buy(t) - use(t)
end-do
```


Loops

- Can nest forall statements

```
forall(t in 1..NT) do
  MaxRef(t) := sum(i in 1..NI) use(i,t) <= MAXREF(t)

  forall(i in 1..NI)
    Inven(i,t) := store(i,t) = store(i,t-1) + buy(i,t) - use(i,t)
end-do
```

Conditions

- May include conditions in sums or loops

```
forall(c in 1..10 | CAP(c) >= 100.0)
  MaxCap(c) :=
    sum(i in 1..10, j in 1..10 | i <> j)
      TECH(i, j, c) * x(i, j, c) <= MAXTECH(c)
```

Mosel statements

- Can extend over several lines and use spaces
- However, a line break acts as an expression terminator
- To continue an expression, it must be cut after a symbol that implies continuation (e.g. `+ - , )`
 - alternatively, when defining the `explterm` option line ends are marked with `';`

Model building style

- You should aim to build a model with sections in this order
 - **constant data**: declare, initialize
 - **all non-constant objects**: declare
 - **variable data**: initialize / input / calculate
 - **decision variables**: create, specify bounds
 - **constraints**: declare, specify
 - **objective**: declare, specify, optimize

Model building style

- In the formulation of (LP, MIP,...) optimization models it is very important to distinguish between
 - **known** values
 - data, parameters, *etc.*
 - and **unknown** values
 - decision variables
- All constraints must be linear expressions of the variables

Model building style

- Suggestion: name objects as follows
 - known values (data) using upper case
 - unknown values (variables) using lower case
 - constraints using mixed case

so that it is easy to distinguish between them, and see that constraints are indeed linear

Model building style

- Variables are **actions** that your model will prescribe
- Use verbs for the names of variables
 - this emphasizes that variables represent 'what to do' decisions
- Indices are the **objects** that the actions are performed on
- Use nouns for the names of indices

Model building style

- Using named index sets/ranges
 - improves the readability of a model
 - makes it easier to apply the model to different sized data sets
 - makes the model easier to maintain
 - may speed up your model

Model building style

- Name constraints
 - helps document them (also describe them with a comment when you declare them)
 - allows Mosel and the Optimizer to produce better error and log messages
 - you cannot obtain solution information or refer to anonymous constraints
 - you cannot change, hide or delete anonymous constraints

Model building style

- Declare objects
 - You don't have to!
 - Allows the compiler to detect syntax errors more easily
 - Mosel's guessed declaration doesn't always work
 - A form of rigour and documentation
 - An opportunity for a descriptive comment

Model building style

- Try to include 'Min' or 'Max' in the name of your objective function
- An objective function called 'Obj' is not very helpful when taken out of context!

Model building style

- Comments are essential for a well written model
- Always use a comment to explain what each parameter, data table, variable, and constraint is for when you declare it
- Add extra comments to explain any complex calculation *etc.*

Model building style

- Comments in Mosel:

```
declarations
  PRODUCTS = 1..NP           ! Set of products
  TIMES = 1..NT              ! Set of time periods
  make: array(PRODUCTS, TIMES) of mpvar
                                ! Amount of p produced in time t
  sell: array(PRODUCTS, TIMES) of mpvar
                                ! Amount of p sold in time t
end-declarations

(! And here is a multi-line
comment !)   forall(t in TIMES)
```



Accessing data sources

- The initializations block
- Sparse arrays
- Run-time parameters
- Using other data sources

Topics

- Text files
- Excel spreadsheets
- ODBC
- Sparse data

Separation of problem logic and data

- Typically, the model logic stays constant once developed, with the data changing each run
- Editing the model can create errors, expose intellectual property, and is impractical for industrial size data
- It makes good sense to fix the model and obtain data from their source



The initializations block

Data input from file: Chess 4

```
uses "mmxprs"

declarations
  PRODS = 1..2                                ! Index range
  DUR, WOOD, PROFIT: array(PRODS) of real      ! Coefficients
  x: array(PRODS) of mpvar                     ! Array of variables
end-declarations

initializations from "chess.dat"               ! Read data from file
  DUR WOOD PROFIT                               !!! chess.dat: PROFIT: [5 20]
end-initializations                             !!!           DUR: [3 2]
                                                !!!           WOOD: [1 3]

sum(i in PRODS) DUR(i)*x(i) <= 160              ! Constraint definition
sum(i in PRODS) WOOD(i)*x(i) <= 200
forall(i in PRODS) x(i) is_integer
maximize(sum(i in PRODS) PROFIT(i)*x(i))        ! Solve the problem
writeln("Solution: ", getobjval)
```

Data file chess.dat

- Every data item/table has a **label**, its identifier
- Single line comments (marked with '!')

```
! Data file for 'chess4.mos'
```

```
DUR:      [3 2]  
WOOD:     [1 3]  
PROFIT:   [5 20]
```

Sparse data format

- Every data entry specified with its index tuple
- Can read data from one labeled data source into several Mosel data tables at once
 - data tables must have identical indices

```
initializations from 'chess.dat'  
  [DUR, WOOD, PROFIT] as 'ChessData'  
end-initializations
```

Sparse data format

- Format of data file with several data values in one labeled data range (use a * for a missing data value)

```
! chess.dat
```

```
ChessData: [  
  (1) [3 1 5]  
  (2) [2 3 20]  
]
```

Writing data out to text files

- You can write out values in an analogous way to reading them in using `initializations to`
- To write out the solution values of variables, or other solution values (slack, activity, dual, reduced cost) you must first put the values into a data table, or alternatively, `use evaluation of`

Writing data out to text files

```
declarations
```

```
  x_sol: array(PRODS) of real
end-declarations
```

```
forall(i in PRODS)
  x_sol(i) := getsol(x(i))
```

```
initializations to 'result.dat'
  x_sol
end-initializations
```

```
! Alternatively:
initializations to 'result.dat'
  evaluation of array(i in PRODS) x(i).sol as 'x_sol'
  evaluation of getobjval as 'obj_sol'
end-initializations
```

Free format text files

```
! Redirect output to 'result.dat' (no screen display), appending to existing contents  
fopen("result.dat", F_OUTPUT+F_APPEND)  
  
forall(i in PRODS)  
    writeln(i, ": ", getsol(x(i))  
  
fclose(F_OUTPUT)           ! Terminate output redirection
```


Free format text files

```
! 'tee:' redirects output to 'result.dat' and also displays on screen/standard output  
fopen("tee:result.dat&", F_OUTPUT+F_APPEND)  
  
forall(i in PRODS)  
    writeln(i, ": ", getsol(x(i))  
  
fclose(F_OUTPUT)           ! Terminate output redirection
```

Project work [C-2]: Arrays and index sets

- Modify the model `chess4.mos` to use indices of type `string`.
- Execute this new model `chess4s.mos` with data set `chess2.dat`.
- Output the solution values to file `sol.dat` using `initializations to`.
- Modify the models further to read the contents of the index set from file (`chess3.dat`, `chess4.dat`).

String indices and data output: Chess 4s

```
uses "mmxprs"

declarations
  PRODS = {"small", "large"}           ! Index set
  DUR, WOOD, PROFIT: array(PRODS) of real ! Coefficients
  x: array(PRODS) of mpvar             ! Array of variables
end-declarations

initializations from "chess2.dat"       ! Read data from file
  DUR WOOD PROFIT
end-initializations

sum(i in PRODS) DUR(i)*x(i) <= 160      ! Constraint definition
sum(i in PRODS) WOOD(i)*x(i) <= 200
forall(i in PRODS) x(i) is_integer
maximize(sum(i in PRODS) PROFIT(i)*x(i))

initializations to "sol.dat"            ! Write solution to file
  evaluation of array(i in PRODS) x(i).sol as 'x'
end-initializations
```



Sparse arrays

Sparse arrays

- Mosel provides a user friendly and efficient means of modeling mathematical programming problems
- Objects such as **sparse arrays** and **variable index sets**, together with efficient loops and sums, allow large scale models to be written easily, and execute quickly

Sparse arrays

- Mosel allows you to use **variable** index sets
 - elements may be assigned to variable index sets throughout the model, *e.g.*, by inputting data from file or using ODBC
 - so the elements may be **defined at run-time**, not compile-time

Sparse arrays

- Declare variable index sets
 - entries are defined later

```
declarations
```

```
  PRODS: set of string
```

```
  TYPE: set of integer
```

```
  PERIODS: range           ! = set of contiguous integers
```

```
end-declarations
```

- Declare sparse arrays (dynamic or hashmap)

```
declarations
```

```
  COST: dynamic array (PRODS, PERIODS) of real
```

```
  HARD: dynamic array (PRODS) of real
```

```
  use: dynamic array (PRODS, PERIODS) of mpvar
```

```
  MinHard: dynamic array (PERIODS) of lincpr
```

```
end-declarations
```

Sparse arrays

- Use sparse arrays
 - to size data tables automatically when the data is read in
 - to initialize the index values automatically when the data is read in
 - to conserve memory when storing sparse data
 - to eliminate index combinations without using conditions each time

Sparse arrays

- Don't use sparse arrays
 - when you can use an ordinary (dense) array instead
 - when storing dense data, and you can size the data table and initialize the indices in some other way (sparse arrays are slower and use more memory than a static array when storing dense data)

Sparse arrays

- Array declaration using variable index sets
 - not the same as explicitly `dynamic` arrays
 - for more efficient handling, Mosel will try to transform these arrays into static arrays after their initialization with data

declarations

COST: `array`(PRODS,PERIODS) of real

HARD: `array`(PRODS) of real

use: `array`(PRODS,PERIODS) of mpvar

MinHard: `array`(PERIODS) of lincptr

end-declarations

Sparse arrays

- Initialize sparse arrays from external sources (text files, databases, ...)
 - must use **sparse format** (= data values + indices)
 - this is so Mosel can work out the values of the indices
 - reading in the data array initializes **both** the **index values** and the **data values** at the same time

Sparse arrays of decision variables

- An entry of a sparse array is only created when a value is assigned to it
- Decision variables don't get created, because you don't assign values to them
- To create decision variables in a sparse array, use the `create` procedure

```
declarations
```

```
PERIODS: range           ! = set of contiguous integers
```

```
COST: dynamic array(PERIODS) of real
```

```
use: dynamic array(PERIODS) of mpvar
```

```
end-declarations
```

```
(...)           ! Read in COST data etc
```

```
forall(t in PERIODS | exists(COST(t))) create(use(t))
```

Sparse arrays of decision variables

- Use the explicit `dynamic` (or `hashmap`) marker in the array declaration if you want to use `create` to control exactly which elements get created



Run-time parameters

Run-time parameters

- Parameters
 - a special type of constant
 - default value may be overridden at run-time

```
parameters  
  DATA_DIR = 'c:/data'  
  DEBUG = true  
  NUM_RECORDS = 1000  
end-parameters
```

Run-time parameters

- The value in the model is used by default
- A different value may be given at run-time
 - in Workbench, an alternative value may be entered after the filename in the *Command* input box of the output pane
 - when running a Mosel model from an application, an alternative value can be set in the parameters string

Run-time parameters

- A `parameters` section must come at the top of the model
 - after any `uses` or `options` statements
 - before any other statements

Run-time parameters

- Parameters are especially useful for passing directories/paths into the model
 - all files referenced in the model should use a directory parameter
 - otherwise, Mosel may not be able to find the file when the model is deployed (the default path differs when run from an application)
 - use '+' to join strings

Run-time parameters

- Specifying directory paths
 - preferably use '/' as directory separator

```
parameters
```

```
  DIR = '.'
```

```
end-parameters
```

```
fopen(DIR+'/cap.dat', F_INPUT)
```

```
...
```

```
fclose(F_INPUT)
```

```
...
```

```
initializations from DIR+'/cost.dat'
```

```
...
```

Project work [C-3]: Run-time parameters

- In models `chess5.mos` and `chess5s.mos` turn the data file name into a run-time parameter.
- Re-run your model `chess5s.mos` with the larger data set `chess3.dat` without changing the filename in the model.

Project work [C-3]:

Run-time parameters

- Setting runtime parameters within Workbench:
 - specify the parameter value after the filename in the *Command* input box of the output pane:

```
chess5s.mos DATAFILE='chess3.dat'
```

- Setting runtime parameters in Visual Studio Code:
 - add the parameter to the `args` list in your `.vscode/launch.json` run configuration:

```
"args": ["DATAFILE='chess3.dat'"]
```

- Runtime parameters from the command line:

```
mosel exe chess5s.mos DATAFILE='chess3.dat'
```

- or:

```
mosel chess5s DATAFILE='chess3.dat'
```

Run-time parameters: Chess 5

```
uses "mmxprs"
parameters
  FILENAME="chess.dat"           ! Name of the data file
end-parameters

declarations
  PRODS: range                   ! Index range
  DUR, WOOD, PROFIT: array(PRODS) of real ! Coefficients
end-declarations

initializations from FILENAME    ! Read data from file
  DUR WOOD PROFIT
end-initializations

declarations
  x: array(PRODS) of mpvar       ! Array of variables
end-declarations

sum(i in PRODS) DUR(i)*x(i) <= 160 ! Constraint definition
sum(i in PRODS) WOOD(i)*x(i) <= 200
forall(i in PRODS) x(i) is_integer
```



Using other data sources

Using other data sources

- The `initializations` block can work with many different data sources and formats thanks to the notion of *I/O drivers*

Using other data sources

- The `initializations` block can work with many different data sources and formats thanks to the notion of *I/O drivers*
- I/O drivers for physical data files: `mmodbc.odbc`, `mmsheet.excel`, `mmoci.oci`, `mmetc.diskdata`

Using other data sources

- The `initializations` block can work with many different data sources and formats thanks to the notion of *I/O drivers*
- I/O drivers for physical data files: `mmodbc.odbc`, `mmsheet.excel`, `mmoci.oci`, `mmetc.diskdata`
- Other drivers available, *e.g.* for data exchange in memory

Using other data sources

- The `initializations` block can work with many different data sources and formats thanks to the notion of *I/O drivers*
- I/O drivers for physical data files: `mmodbc.odbc`, `mmsheet.excel`, `mmoci.oci`, `mmetc.diskdata`
- Other drivers available, *e.g.* for data exchange in memory
- Change of the data source = change of the I/O driver, no other modifications to your model

Data exchange with spreadsheets

- Software-specific driver *excel* for MS Excel
 - use the *mmsheet.excel* driver in `initializations` blocks
 - no driver setup required (works with standard Excel installation)
 - add the prefix `"mmsheet.excel: "` to the file name
- Module *mmsheet* also defines alternative, platform-independent spreadsheet drivers (*xls*, *xlsx*, *csv*) that do not rely on Excel
 - usage:
`"mmsheet.xlsx:mydatafile.xlsx" "mmsheet.csv:mydatafile.csv"`

Data exchange with MS Excel

```
initializations from 'mmsheet.xls:chess.xls'  
  [DUR, WOOD, PROFIT] as 'ChessData'    ! 'ChessData': range area is B4:E8  
end-initializations  
  
initializations to 'mmsheet.xls:chess.xls'  
  x_sol as 'grow;ChessSol'              ! 'ChessSol': first row of output area: H5:I5  
end-initializations
```

	A	B	C	D	E	F	G	H	I	J
1										
2										
3		ChessData						ChessSol		
4		ITEM	DUR	WOOD	PROFIT			ITEM	VALUE	
5		small	3	1	5					
6		medium	2	2	12					
7		large	2	3	20					
8		extra	3	6	40					
9										
10										

Referencing Excel ranges

- Named ranges
 - select the desired area with your mouse
 - MS Excel 2010 and newer: use the menu *Formulas* » *Define Name* to enter the range name
- Direct reference to worksheet cells
 - standard notation: [DUR, WOOD, PROFIT] as '[Sheet1\$B5:E8]'
 - row-column notation: [DUR, WOOD, PROFIT] as '[Sheet1\$R5C2:R8C5]'
- Select columns within a range
 - using numbers: [PROFIT,DUR] as 'ChessData(#1,#4,#2)'
 - using header text: [PROFIT,DUR] as 'skip;[Sheet1\$B4:E8](ITEM,PROFIT,DUR)'

Spreadsheet drivers

- Module *mmsheet* defines alternative spreadsheet drivers: *xls*, *xlsx* and *csv*

	<i>excel</i>	<i>xls/xlsx</i>	<i>csv</i>
File type	physical file	physical file	extended file
Supported platforms	Windows	Windows, Linux, Mac	all Xpress platforms
Requirements	Excel + open interactive session	none, can be used remotely	none, can be used remotely
File creation for output	no	yes	yes
Output writing mechanism	on-screen display without saving if application running, otherwise data saved into file	data saved into file	data saved into file
Named ranges	yes	yes	no
Multiple worksheets	yes	yes	no
VBA Macros	yes	no	no

Data transfer using ODBC

- Use the *mmodbc* module
 - use the *odbc* driver in `initializations` blocks, or
 - write out the corresponding SQL commands:
 - set up an ODBC data connection to the specific data source
 - input data using SQL statements
 - disconnect
- The *mmodbc* module includes SQLite (lightweight filebased database) on all platforms supported by Mosel
 - no additional software installation or configuration required

Reading data via ODBC

- SQLite database ('ChessData' = data table):

```
initializations from 'mmodbc.odbc:debug;chess.sqlite'  
  [DUR, WOOD, PROFIT] as 'ChessData'  
end-initializations
```

Reading data via ODBC

- SQLite database ('ChessData' = data table):

```
initializations from 'mmodbc.odbc:debug;chess.sqlite'  
  [DUR, WOOD, PROFIT] as 'ChessData'  
end-initializations
```

- Access database:

```
initializations from 'mmodbc.odbc:debug;chess.accdb'  
...
```

- Note that if you use ODBC to access components of MS Office (such as Access), the bitness of your Xpress installation must match the bitness of MS Office

Data export to a database

```
initializations to 'mmodbc.odbc:debug;chess.sqlite'  
  x_sol as 'ChessSol'  
end-initializations
```

- Before every new run, delete the data from the previous run in the destination range/table
- Otherwise the new results will either be appended to the existing ones or, if 'PRODS' has been defined as key field in a database, the insertion will fail

Data exchange with Oracle

- Software-specific driver *oci* for Oracle databases
 - use *mmoci* module
 - setup: Oracle's Instant Client package must be installed on the machine running the Mosel model
 - in `initializations` blocks replace
"mmodbc.odbc:" by "mmoci.oci:" in the preceding examples
 - supports SQL statements (replace the prefix `SQL` by `OCI`)

SQL

- The I/O driver *odbc* generates automatically the SQL commands required to connect to the database/spreadsheet
- For advanced uses, module *mmodbc* also defines most standard SQL commands directly for the Mosel language

SQL

- Data input using SQL statements

```
uses 'mmodbc'
```

```
setparam('SQLverbose', true)  
setparam('SQLdebug', true)  
SQLconnect('chess.mdb')  
SQLexecute('select * from ChessData', [DUR, WOOD, PROFIT])  
SQLexecute('delete from ChessSol')  
SQLdisconnect
```

- Data output using SQL:
 - create a new data table and write out data

```
SQLexecute('create table ChessSol (PName VARCHAR(20), Sol DOUBLE)')  
SQLexecute('insert into ChessSol (PName,Sol) values (?,?)', x_sol)
```

- or may use SQL update (if supported by database)

```
SQLupdate('select PName,Sol from ChessSol', x_sol)
```

SQL

- Using SQL, much more is possible...

```
SQLexecute('select * from Cost where Legs >= 4', COST)
```

```
SQLexecute('select count(*) from PRODS', SIZE)
```

```
SQLexecute('select CostS,CostM,CostL from CostData  
where (CostS<2 and CostL>10) order by Products,ProdCode', PRICE)
```

Project work [C-4]: Spreadsheets and databases

- Re-run your model `chess5.mos` with the Excel file `chess.xls`
- Re-run your model `chess5.mos` with the SQLite file `chess.sqlite`

Summary

- We have seen that it is possible to completely separate the data and the model
- The model specifies the logic of the problem, without any reference to its size
- The model can be applied to any data instance, simply by providing data files

Reference material

- Refer to the [Mosel User Guide](#) for a detailed introduction to working with Mosel.
- The book [Applications of optimization with Xpress-MP](#) provides a large collection of examples models from different application areas.
 - see files under examples/mosel/AplBook
- See the whitepaper [Using ODBC and other database interfaces with Mosel](#) for further detail on data handling.
- The whitepaper [General file handling in Mosel](#) gives an overview of available I/O drivers and their uses.



Advanced modeling topics

- Integer Programming entities supported in Xpress
- Modeling with binary variables



Integer Programming entities
supported in Xpress

Integer Programming entities supported in Xpress

- Binary variables

- can take either the value 0 or the value 1 (do/ don't do variables)
- model logical conditions

```
x(4) is_binary
```

Integer Programming entities supported in Xpress

- Integer variables

- can take only integer values
- used where the underlying decision variable really has to take on a whole number value for the optimal solution to make sense

`x(7) is_integer`

Integer Programming entities supported in Xpress

- Semi-continuous variables

- can take either the value 0, or a value between some lower limit and upper limit
- help model situations where if a variable is to be used at all, it has to be used at some minimum level

`x(2) is_semcont 6    ! A 'hole' between 0 and 6, then continuous`

Integer Programming entities supported in Xpress

- Semi-continuous integer variables

- can take either the value 0, or an integer value between some lower limit and upper limit
- help model situations where if a variable is to be used at all, it has to be used at some minimum level, and has to be integer

`x(3) is_semint 7` *! A 'hole' between 0 and 7, then integer*

Integer Programming entities supported in Xpress

- **Special Ordered Sets of type one (SOS1)**
 - an ordered set of variables at most one of which can take a non-zero value
 - single choice among several possibilities
- **Special Ordered Sets of type two (SOS2)**
 - an ordered set of variables, of which at most two can be non-zero, and if two are non-zero these must be consecutive in their ordering
 - *e.g.* approximation of non-linear functions with a piecewise linear function

SOS definition

- WEIGHT array determines the ordering of the variables:

`MYSOS:= sum(i in IRng) WEIGHT(i)*x(i) is_sosX`

where `is_sosX` is either `is_sos1` or `is_sos2`

SOS definition

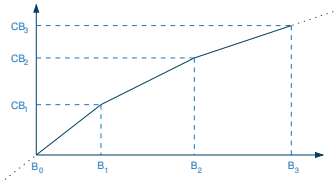
- Formulation alternative: set S of set members, linear constraint L with ordering coefficients (= **reference row entries**):

`makesos1 (S, L) ; makesos2 (S, L)`

- must be used if the coefficient `WEIGHT (i)` of an intended set member is zero
- **Note:** the ordering coefficients must all be distinct (or else they are not doing their job of supplying an order!)

Integer Programming entities supported in Xpress

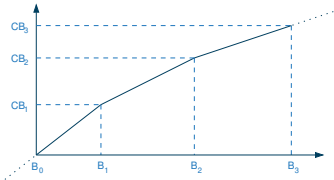
- Piecewise linear expressions can be used in place of SOS2 formulations
- Definition of piecewise linear expressions in Mosel:



- *For all forms of `pwlin` the curve continues beyond the first and last breakpoint, prolongating the first and last segments to infinity*

Integer Programming entities supported in Xpress

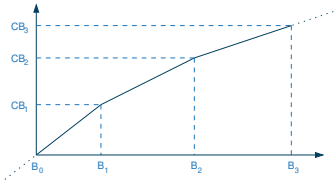
- Piecewise linear expressions can be used in place of SOS2 formulations
- Definition of piecewise linear expressions in Mosel:



- For all forms of `pwlin` the curve continues beyond the first and last breakpoint, prolongating the first and last segments to infinity
- specification as **list of slopes** with associated intervals:
implementation with Mosel: only points of slope changes are specified, start value is 0:
`y = pwlin(x, union(i in 1..2) [B(i)], union(i in 1..3) [SLOPE(i)])`

Integer Programming entities supported in Xpress

- Piecewise linear expressions can be used in place of SOS2 formulations
- Definition of piecewise linear expressions in Mosel:



- *For all forms of `pwlin` the curve continues beyond the first and last breakpoint, prolongating the first and last segments to infinity*

- specification as **list of slopes** with associated intervals:
implementation with Mosel: only points of slope changes are specified, start value is 0:

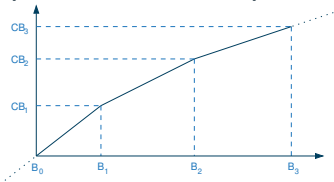
```
y = pwlin(x, union(i in 1..2) [B(i)], union(i in 1..3) [SLOPE(i)])
```

- specification as **list of piecewise linear segments** with associated intervals:

```
y = pwlin(union(i in 1..3) [pws(B(i-1), CB(i-1)+SLOPE(i)*(x-B(i-1)))])
```

Integer Programming entities supported in Xpress

- Piecewise linear expressions can be used in place of SOS2 formulations
- Definition of piecewise linear expressions in Mosel:



- *For all forms of `pwlin` the curve continues beyond the first and last breakpoint, prolongating the first and last segments to infinity*

- specification as **list of slopes** with associated intervals:
implementation with Mosel: only points of slope changes are specified, start value is 0:

```
y = pwlin(x, union(i in 1..2) [B(i)], union(i in 1..3) [SLOPE(i)])
```

- specification as **list of piecewise linear segments** with associated intervals:

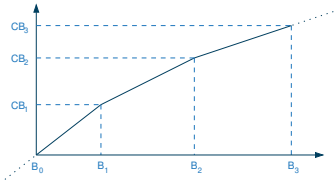
```
y = pwlin(union(i in 1..3) [pws(B(i-1), CB(i-1)+SLOPE(i)*(x-B(i-1)))])
```

- specification as **list of points**:

```
y = pwlin(x, union(i in 0..3) [B(i), CB(i)])
```

Integer Programming entities supported in Xpress

- Piecewise linear expressions can be used in place of SOS2 formulations
- Definition of piecewise linear expressions in Mosel:



- *For all forms of `pwlin` the curve continues beyond the first and last breakpoint, prolongating the first and last segments to infinity*

- specification as **list of slopes** with associated intervals:
implementation with Mosel: only points of slope changes are specified, start value is 0:

```
y = pwlin(x, union(i in 1..2) [B(i)], union(i in 1..3) [SLOPE(i)])
```

- specification as **list of piecewise linear segments** with associated intervals:

```
y = pwlin(union(i in 1..3) [pws(B(i-1), CB(i-1)+SLOPE(i)*(x-B(i-1)))])
```

- specification as **list of points**:

```
y = pwlin(x, union(i in 0..3) [B(i), CB(i)])
```


Integer Programming entities supported in Xpress

- **Indicator constraints** are logic constraints that associate a binary variable b with a linear (or nonlinear) constraint C . They take one of these two forms
'if $b = 1$ then C ', in symbols: $b \rightarrow C$,
or
'if $b = 0$ then C ', in symbols: $\text{not}(b) \rightarrow C$
- Xpress Optimizer accepts indicator constraints in MI(NL)P models.

Indicator constraints

- Definition of indicator constraints in Mosel:

```
declarations
  R=1..2
  x,b: array(R) of mpvar
end-declarations

forall(i in R) b(i) is_binary  ! Variables for indicator constraints
C:= x(2)<=5                    ! Auxiliary constraint definition
indicator(1, b(1), x(1)+x(2)>=12)  ! b(1)=1 -> x(1)+x(2)>=12
indicator(-1, b(2), C)           ! b(2)=0 -> x(2)<=5
C:=0                             ! Delete auxiliary constraint def.
```

Integer Programming entities supported in Xpress

- General constraints
 - specific constraint relations that are recognized by MIP solvers
 - **piecewise linear**: can be used in place of SOS2 formulations
 - **absolute** value, **minimum** value, **maximum** value of discrete or continuous decision variables
 - logical constraints: '**and**' and '**or**' over binary variables

Integer Programming entities supported in Xpress

- General constraints
 - specific constraint relations that are recognized by MIP solvers
 - **piecewise linear**: can be used in place of SOS2 formulations
 - **absolute** value, **minimum** value, **maximum** value of discrete or continuous decision variables
 - logical constraints: '**and**' and '**or**' over binary variables
 - direct statement of (nonlinear) relations without any need for reformulation via auxiliary binary variables on the modeling level
 - solver chooses the most appropriate representation and applies presolving to these constructs

Integer Programming entities supported in Xpress

- General constraints

- specific constraint relations that are recognized by MIP solvers
 - **piecewise linear**: can be used in place of SOS2 formulations
 - **absolute** value, **minimum** value, **maximum** value of discrete or continuous decision variables
 - logical constraints: '**and**' and '**or**' over binary variables
- direct statement of (nonlinear) relations without any need for reformulation via auxiliary binary variables on the modeling level
- solver chooses the most appropriate representation and applies presolving to these constructs

- Mosel implementation:

- modeling constructs `pwlin`, `fmin`, `fmax`, `abs` (of type `nlctr`) for the formulation of MIP require `mmxnlp` but will result in MIP models if they only involve linear expressions
- a second form of logical constraints (type `logctr`) based on Boolean variables (type `boolvar`) is implemented via `mmxprs`

```
uses "mmxnlp"
```

```
abs(x(1)-2*x(2)) <= 10
```

```
MinCtr:= fmin(union(i in R) [x(i)]) >= 5
```

```
y = fmax(x(3), 20, 2*x(1)-z)
```

```
! Absolute value constraint
```

```
! Minimum value constraint
```

```
! Maximum value constraint
```

Absolute value: MIP formulations

Example: $v = |x_1 - x_2|$

- Formulation via binary variables:

```
uses "mmxprs"
declarations
  C: real                                ! Must know bounds on x(i): assume 0 <= x(i) <= C
  x: array(1..2) of mpvar
  v, d: mpvar
  C1a, C1b, C2a, C2b, C3a, C3b: linctr
end-declarations

d is_binary                                ! d = 0 if x(1)-x(2) is positive
                                           ! d = 1 if x(2)-x(1) is positive

C1a:= x(1) <= C                            ! x(1) >= 0 automatically
C1b:= x(2) <= C                            ! x(2) >= 0 automatically
C2a:= v >= x(1) - x(2)
C2b:= v <= x(1) - x(2) + 2*C*d
C3a:= v >= x(2) - x(1)
C3b:= v <= x(2) - x(1) + 2*C*(1-d)
```

Absolute value: MIP formulations

Example: $v = |x_1 - x_2|$

- Formulation via indicator constraints

```
uses "mmxprs"  
declarations  
  x: array(1..2) of mpvar  
  v, d: mpvar  
  C2a, C3a: lincstr  
  C2b, C3b: logctr  
end-declarations
```

```
d is_binary  
C2a:= v >= x(1) - x(2)  
C3a:= v >= x(2) - x(1)  
C2b:= indicator(-1, d, v <= x(1) - x(2))  
C3b:= indicator(1, d, v <= x(2) - x(1))
```

! d = 0 -> x(1)-x(2) is positive

! d = 1 -> x(2)-x(1) is positive

Absolute value: MIP formulations

Example: $v = |x_1 - x_2|$

- Formulation as general constraint

```
uses "mmxnlp"  
declarations  
  x: array(1..2) of mpvar  
  v: mpvar  
  C2: nlctr  
end-declarations  
  
C2:= v = abs(x(1) - x(2))
```


Maximum value: MIP formulations

Example: $u = \max(x_1, x_2, x_3)$

- Formulation via binary variables:

```
uses "mmxprs"
declarations
  R = 1..3
  L,U: array(R) of real           ! Must know bounds on x(i): assume L(i) <= x(i) <= U(i)
  x,d: array(R) of mpvar
  u: mpvar
end-declarations

forall(i in R) d(i) is_binary ! d(i) = 1 iff x(i) is maximum
Umax:= max(i in R) U(i)       ! Largest upper bound

forall(i in R) do
  L(i) <= x(i); x(i) <= U(i)
  u >= x(i)
  u <= x(i) + (Umax-L(i))*(1-d(i))
end-do

sum(i in R) d(i) >= 1
```

Maximum value: MIP formulations

Example: $u = \max(x_1, x_2, x_3)$

- Formulation via indicator constraints

```
uses "mmxprs"  
declarations  
  R = 1..3  
  x,d: array(R) of mpvar  
  u: mpvar  
end-declarations  
  
forall(i in R) d(i) is_binary           ! d(i) = 1 iff x(i) is maximum  
  
forall(i in R) do  
  u >= x(i)  
  indicator(1, d(i), u <= x(i))       ! d(i) = 1 -> u <= x(i)  
end-do  
  
sum(i in R) d(i) >= 1
```

Maximum value: MIP formulations

Example: $u = \max(x_1, x_2, x_3)$

- Formulation as general constraint

```
uses "mmxnlp"  
declarations  
  R = 1..3  
  x: array(R) of mpvar  
  u: mpvar  
end-declarations  
  
u = fmax(union(i in R) [x(i)] )
```

Logical constraints over Boolean variables

- Mosel defines the type `boolvar` for the formulation of logical constraints 'and', 'or', 'not'
 - the resulting constraints are directly sent to the solver as general constraints on binary variables $\Rightarrow$ MIP model

```
public declarations
```

```
  R=1..5
```

```
  bv: array(R) of boolvar
```

```
  LC1, LC2: logctr
```

```
end-declarations
```

```
bv(1) and not bv(5)           ! Simple clause, same as: bv(1) and (not bv(5)) = true
```

```
bv(3) = (not bv(4))           ! Association of clauses
```

```
! Defining a logic expression (not recorded in the constraint store)
```

```
LC1:= and(i in 1..3) bv(i) or and(i in 4..5) not bv(i)
```

```
LC1:= LC1=true                 ! Turn expression into a constraint
```

```
! A named logic expression (this defines a constraint)
```

```
LC2:= (or(i in 1..3) not bv(i)) = false
```



Modeling with binary variables

Logical conditions

- Projects A, B, C, D
- Binary variables a, b, c, d
 - do at most 3 projects: $a + b + c + d \leq 3$
 - must do D if A done: $d \geq a$
 - can only do C if both A and B done:
 $c \leq (a + b)/2$
 $c \leq a, c \leq b$

Disjunctions

- Either $5 \leq x \leq 10$
or $80 \leq x \leq 100$
- Introduce a new variable:
ifupper: 0 if $5 \leq x \leq 10$; 1 if $80 \leq x \leq 100$
 $x \leq 10 + (100 - 10) \cdot \textit{ifupper}$ [1]
 $x \geq 5 + (80 - 5) \cdot \textit{ifupper}$ [2]

Disjunctions

- Either $5 \leq x \leq 10$
or $80 \leq x \leq 100$
- Introduce a new variable:
ifupper: 0 if $5 \leq x \leq 10$; 1 if $80 \leq x \leq 100$
 $x \leq 10 + (100 - 10) \cdot \textit{ifupper}$ [1]
 $x \geq 5 + (80 - 5) \cdot \textit{ifupper}$ [2]
- Generalizes to
either $5 \leq \sum_i A_i x_i \leq 10$
or $80 \leq \sum_i A_i x_i \leq 100$
 $\sum_i A_i x_i \leq 10 + 90 \cdot \textit{ifupper}$ [1]
 $\sum_i A_i x_i \geq 5 + 75 \cdot \textit{ifupper}$ [2]

Project work [K-3]: Logical constraints

- Take a look at the capital budgeting model in `capbgt.mos`: the objective is to determine the most profitable choice among 8 possible projects, subject to limited resources (personnel and capital)
- Formulate the following additional constraints:
 1. P1 can only be done if P2 is done
 2. P1 can only be done if P3 and P6 are done
 3. It is not possible to do both P5 and P6
 4. Either P1 and P2 must be done or P3 and P4 (but not both pairs).
- Try to find alternative formulation options for constraint 4.
 - Compare the matrices resulting from different implementations (using `writetprob` with option "I").

Solution

! p1 can only be done if p2 is done

`select(2) >= select(1)`

! p1 can only be done if p3 and p6 are done

`(select(3) + select(6))/2 >= select(1)`

! Or:

`select(3) >= select(1)`

`select(6) >= select(1)`

! It is not possible to do both p5 and p6

`select(5) + select(6) <= 1`

! Either p1 and p2 must be done or p3 and p4 (but not both pairs).

`select(1) = select(2); select(3) = select(4)`

`select(1) + select(2) = 2 - (select(3) + select(4))`

Reference material

- See the [MIP formulations and linearizations](#) quick reference guide for an overview of MIP variable types (including indicator constraints) and their use.



Programming language features

- Selections
- Loops
- Functions and procedures
- Data structures

Mosel:

A programming environment

- Selections
- Loops
- Set operations
- Subroutines
- Data structures



Selections

- if

```
if A >= 20 then
  x <= 7
elif A <= 10 then
  x >= 35
else
  x = 0
end-if
```

- case

```
case A of
  -1000..10 : x >= 35
  20..1000 : x <= 7
  12, 15 : x = 1
  else : x = 0
end-case
```

- **Try it yourself:** open `examples/select.mos`



Loops

- forall [do]
- while [do]
- repeat until

Example: Prime numbers

- Implements the ‘Sieve of Eratosthenes’.

$SNumbers = \{2, \dots, L\}$

$n := 2$

repeat

 while ($n \notin SNumbers$) $n := n + 1$

$SPrime := SPrime \cup \{n\}$

$i := n$

 while ($i \leq L$)

$SNumbers := SNumbers \setminus \{i\}$

$i := i + n$

until $SNumbers = \{\}$

Example: Prime numbers

```
model Prime
parameters
  LIMIT=100                                ! Search for prime numbers in 2..LIMIT
end-parameters

declarations
  SNumbers: set of integer                 ! Set of numbers to be checked
  SPrime: set of integer                   ! Set of prime numbers
end-declarations

SNumbers:={2..LIMIT}
writeln("Prime numbers between 2 and ", LIMIT, ":")
```

Example: Prime numbers

```
n:=2
repeat
  while (not(n in SNumbers)) n+=1
  SPrime += {n}                ! n is a prime number
  i:=n
  while (i<=LIMIT) do         ! Remove n and all its multiples
    SNumbers-= {i}
    i+=n
  end-do
until SNumbers={}

writeln(SPrime)
writeln(" (", getsize(SPrime), " prime numbers.)")
end-model
```

Operations on sets

- Set operators include
 - union: +
 - intersection: *
 - difference: -
- Logical expressions using sets include
 - subset: `Set1 <= Set2`
 - superset: `Set1 >= Set2`
 - equals: `Set1 = Set2`
 - not equals: `Set1 <>Set2`
 - element of: `'Oil5' in Set1`
 - not element of: `'Oil5' not in Set1`



Functions and procedures

Functions and procedures

- Similar structure as `model`, including the `declarations` blocks
- Terminated by `end-function` or `end-procedure`
- Function defines `returned` with its return value
- `forward declaration`
- Overloading possible (each version with a different number or types of arguments)

Example: Quick Sort algorithm

1. Choose a middle value v for partitioning (here: $v = (\textit{min} + \textit{max})/2$)
2. Divide the list into two parts 'left' (all elements $x < v$) and 'right' (all elements $x > v$)
3. Repeat from 1. for lists 'left' and 'right'

Example: Quick Sort algorithm

```
model "Quick Sort"
parameters
    LIM=50
end-parameters

! Declare procedures that are defined later
forward procedure swap(L:array(range) of integer, i,j:integer)
forward procedure qsort(L:array(range) of integer)
forward procedure qsort(L:array(range) of integer, s,e:integer)

declarations
    T:array(1..LIM) of integer
end-declarations

! Generate randomly an array of numbers
forall(i in 1..LIM) T(i):=round(.5+random*LIM)
writeln(T) ! Print the unsorted array

qsort(T) ! Sort the array
writeln(T) ! Print the sorted array
```

- **Try it yourself:** open `examples/mosel/Programming/qsort.mos` in your Xpress installation

Example: Quick Sort algorithm

```
model "Quick Sort"
parameters
    LIM=50
end-parameters

declarations                ! Declare procedures that are defined later
    procedure swap(L:array(range) of integer, i,j:integer)
    procedure qsort(L:array(range) of integer)
    procedure qsort(L:array(range) of integer, s,e:integer)

    T:array(1..LIM) of integer
end-declarations

                                ! Generate randomly an array of numbers
forall(i in 1..LIM) T(i):=round(.5+random*LIM)
writeln(T)                      ! Print the unsorted array

qsort(T)                       ! Sort the array
writeln(T)                     ! Print the sorted array
```

Example: Quick Sort algorithm

! Start of the sorting process

```
procedure qsort(L:array(r:range) of integer)
  qsort(L,getfirst(r),getlast(r))
end-procedure
```

! Sorting routine

```
procedure qsort(L:array(range) of integer, s,e:integer)
  v:=L((s+e) div 2)
  i:=s; j:=e
  repeat
    while(L(i)<v) i+=1
    while(L(j)>v) j-=1
    if i<j then
      swap(L,i,j)
      i+=1; j-=1
    end-if
  until i>=j
  if j<e and s<j: qsort(L,s,j)
  if i>s and i<e: qsort(L,i,e)
end-procedure
```

Example: Quick Sort algorithm

```
! Swap the positions of two numbers in an array  
procedure swap(L:array(range) of integer, i,j:integer)  
  k:=L(i)  
  L(i):=L(j)  
  L(j):=k  
end-procedure  
  
end-model
```



Data structures

Data structures

- array
- set

Data structures

- array
- set
- list
- record

Data structures

- array
- set
- list
- record
- ... and any combinations thereof, *e.g.*,

S: set of list of integer

A: array(range) of set of real

List

- Collection of objects of the same type
- May contain the same element several times
- Order of list elements is specified by construction
- Handling: `cuthead`, `splittail`, `reverse`...

```
declarations
```

```
  L: list of integer
```

```
  M: array(range) of list of string
```

```
end-declarations
```

```
L:= [1,2,3,4,5]
```

```
M:= (2..4)[['A','B','C'], ['D','E'], ['F','G','H','I']]
```

Record

- Finite collection of objects of any type
- Each component of a record is called a 'field' and is characterized by its name and its type

```
declarations
  ARC: array (ARCSET:range) of record
    Source, Sink: string      ! Source and sink of arc
    Cost: real                ! Cost coefficient
  end-record
end-declarations
```

```
ARC(1).Source:= "B"
```

```
ARC(3).Cost:= 1.5
```

User types

- Treated in the same way as the predefined types of the Mosel language
- New types are defined in `declarations` blocks by specifying a type name, followed by `=`, and the definition of the type

```
declarations
  myreal = real
  myarray = array(1..10) of myreal
  COST: myarray
end-declarations
```

User types

- Typical uses
 - shorthand for repetitions in declarations
 - naming records

```
declarations
  arc = record
    Source, Sink: string      ! Source and sink of arc
    Cost: real                ! Cost coefficient
  end-record
  A: arc
  ARC: array(ARCSET:range) of arc
end-declarations
```

Summary: Language features

- **Data structures:** array, set, list, record
- **Selections:** if-then-[elif-then]-[else], case
- **Loops:** forall-[do], while-[do], repeat-until
- **Operators:**
 - standard arithmetic operators
 - aggregate operators (sum, prod, and, or, min, max, union, intersection)
 - set operators
- **Subroutines:** functions, procedures
(forward declaration, overloading)

Reference material

- See Part 'II. Advanced language features' of the [Mosel User Guide](#) for a comprehensive collection of Mosel model examples demonstrating programming tasks.



Programming solution algorithms

Mosel:

A solving environment

- No separation between 'modeling statements' and 'solving statements'
- Programming facilities for pre/postprocessing, algorithms
- Principle of incrementality
- Not solver-specific
- Possibility of interaction with solver(s)

Example: Variable fixing heuristic

- Consider the following heuristic for solving problems with binary or integer variables
 - solve the linear relaxation
 - for each binary (or integer) variable, check the solution value: if it is integer valued, fix the variable
 - solve the resulting MIP problem (with the remaining unfixed MIP variables)

Example: Variable fixing heuristic

- Consider the following heuristic for solving problems with binary or integer variables
 - solve the linear relaxation
 - for each binary (or integer) variable, check the solution value: if it is integer valued, fix the variable
 - solve the resulting MIP problem (with the remaining unfixed MIP variables)
- Can you see why this heuristic may fail to find the true optimal solution?

Example: Variable fixing heuristic

- Any MIP solution of the modified problem is a feasible MIP solution of the original problem
- However, it is possible that the modified problem has no feasible MIP solutions, even if the original problem is feasible

Example: Variable fixing heuristic

- Any MIP solution of the modified problem is a feasible MIP solution of the original problem
- However, it is possible that the modified problem has no feasible MIP solutions, even if the original problem is feasible
- $\Rightarrow$ Extended heuristic:
 - solve the problem using the fixing heuristic
 - if a solution was found, store it and record the objective value
 - restore the original problem
 - if an objective value was recorded, use it as a cut-off
 - solve the original MIP problem

Example: Variable fixing heuristic

- Any MIP solution of the modified problem is a feasible MIP solution of the original problem
- However, it is possible that the modified problem has no feasible MIP solutions, even if the original problem is feasible
- $\Rightarrow$ Extended heuristic:
 - solve the problem using the fixing heuristic
 - if a solution was found, store it and record the objective value
 - restore the original problem
 - if an objective value was recorded, use it as a cut-off
 - solve the original MIP problem
- Use the solution found by the heuristic to reduce the size of the second MIP search
 - if the heuristic finds a good solution quickly, the overall search time can be reduced considerably

Example: Variable fixing heuristic (implementation features)

- Performing partial optimizations

- solve the LP relaxation of a MIP

```
setparam("XPRS_MIPSTOPSTAGE", XPRS_MIPSTOPSTAGE_INITIALRELAXATION)  
minimize(MinCost)
```

- solve the remaining MIP problem

```
setparam("XPRS_MIPSTOPSTAGE", XPRS_MIPSTOPSTAGE_NONE)  
minimize(XPRS_CONT, MinCost)
```

- Fixing variables and changing bounds

- variables can be fixed by changing the bounds

```
setub(x(i,j), 0)  
setlb(x(i,j), 1)
```

- Program split into several source files (via `include`)

Example: Variable fixing heuristic (main file)



Note: See `examples/mosel/Solving/fixbv.mos` in your Xpress installation

```
model Coco
  uses "mmxprs"

  include "fixbv_pb.mos"           ! Model definition
  include "fixbv_solve.mos"       ! Algorithm

  solution:=solve
  writeln("The objective value is: ", solution)

end-model
```

Example: Variable fixing heuristic (model)

```
declarations
  RF=1..2                ! Range of factories (f)
  RT=1..4                ! Range of time periods (t)
  (...)
  openm: array(RF,RT) of mpvar
end-declarations

(...)
forall(f in RF,t in 1..NT-1) Closed(f,t):= openm(f,t+1) <= openm(f,t)
forall(f in RF,t in RT) openm(f,t) is_binary
```


Example: Variable fixing heuristic (algorithm)

```
function solve:real
  declarations
    osol: array(RF,1..2) of real
    bas: basis
  end-declarations

  setparam("XPRS_PRESOLVE",0)
  setparam("zerotol", 5.0E-4)      ! Set Mosel comparison tolerance
  setparam("XPRS_MIPSTOPSTAGE", XPRS_MIPSTOPSTAGE_INITIALRELAXATION)
  maximize(MaxProfit)             ! Solve the root LP
  savebasis(bas)                  ! Save the basis

  forall(f in RF, t in 1..2) do   ! Fix some binary variables
    osol(f,t) := getsol(openm(f,t))
    if osol(f,t) = 0 then
      setub(openm(f,t), 0.0)
    elif osol(f,t) = 1 then
      setlb(openm(f,t), 1.0)
    end-if
  end-do
```

Example: Variable fixing heuristic (algorithm)

```
setparam("XPRS_MIPSTOPSTAGE", XPRS_MIPSTOPSTAGE_NONE)
maximize(XPRS_CONT,MaxProfit) ! Solve modified problem
ifgsol:=false
if getprobat=XPRS_OPT then ! If a solution was found
    ifgsol:=true
    solval:=getobjval ! Save solution value
end-if

forall(f in RF, t in 1..2) ! Reset variable bounds
    if((osol(f,t) = 0) or (osol(f,t) = 1)) then
        setlb(openm(f,t), 0.0); setub(openm(f,t), 1.0)
    end-if

loadbasis(bas) ! Load previously saved basis
if ifgsol: ! Set cutoff to best known solution
    setparam("XPRS_MIPABSCUTOFF", solval)
maximize(MaxProfit) ! Solve original problem
returned:= if(getprobat=XPRS_OPT, getobjval, solval)
end-function
```

Implementing heuristics

- It is also possible to save / load complete MIP solutions

```
declarations
  mipsols: mpsol
end-declarations
...          ! Define & solve a MIP
savemipsol(mipsols)
...          ! Modify & reload the MIP
if loadmipsol(mipsols) <> 0 then
  writeln("Problem with loading MIP solution")
end-if
...          ! Solve the MIP problem
```

Implementing heuristics

- Can provide incomplete or infeasible solutions $\Rightarrow$ use addmipsol

```
declarations
  mipsols: array(set of mpvar) of real
end-declarations

...          ! Generate a heuristic solution
mipsols(x(i)):= Heur_sol_val          ! Set solution values
...          ! (Re)load the MIP
addmipsol("SolutionID", mipsols)
setcallback(XPRS_CB_SOLNOTIFY, ->myheurlog) ! Optional callback
...          ! Solve the MIP problem

procedure myheurlog(id:string, status:integer)
  writeln("Optimiser loaded solution ", id, " status=", status)
end-procedure
```

Implementing heuristics

- MIP local search heuristics exploit feasible solutions loaded into the Optimizer
- Parameter setting to reinforce use of these heuristics (not impacted by **HEUREMPHASIS** set to 0):

```
! Exploit user solution in local search  
! (-1:automatic, 0:disable, 1-3:increasingly higher effort)  
setparam("XPRS_USERSOLHEURISTIC", 3)
```

Example: Variable fixing heuristic (alternative alg.)

```
setparam("XPRS_MIPSTOPSTAGE", XPRS_MIPSTOPSTAGE_NONE)
maximize(XPRS_CONT,MaxProfit) ! Solve modified problem
ifgsol:=false
if getprobat=XPRS_OPT then ! If a solution was found
  ifgsol:=true
  solval:=getobjval ! Save solution value
  savemipsol(heurSol) ! Save the solution
end-if

forall(f in RF, t in 1..2) ! Reset variable bounds
  if((osol(f,t) = 0) or (osol(f,t) = 1)) then
    setlb(openm(f,t), 0.0); setub(openm(f,t), 1.0)
  end-if

loadbasis(bas) ! Load previously saved basis
if ifgsol: ! Load the heuristic solution
  addmipsol("startsol", heurSol)
maximize(MaxProfit) ! Solve original problem
returned:= if(getprobat=XPRS_OPT, getobjval, solval)
end-function
```

Reference material

- A Mosel implementation of the variable fixing heuristic is discussed in Chapter 8. *Heuristics* of the [Getting Started](#) manual.



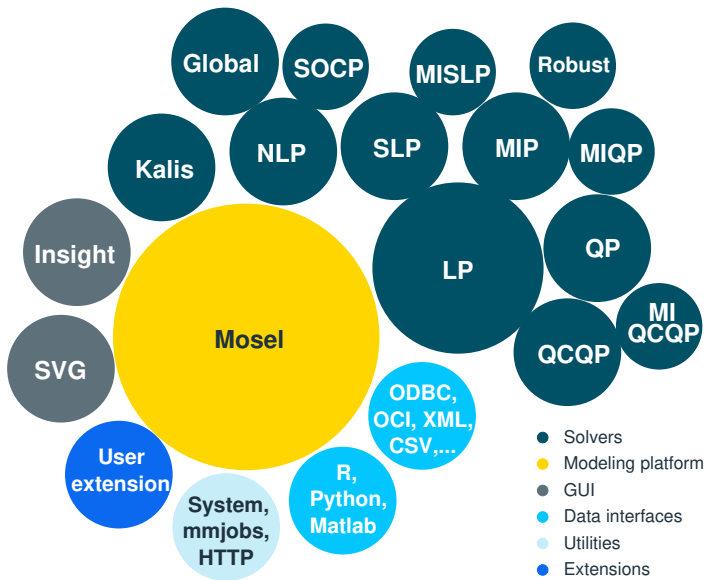
Mosel modules and packages

- Mosel: A modular environment
- Module features: solvers
- Interfacing with other languages
- Drawing user graphs

Mosel: A modular environment

- Open architecture:
 - possibility to define language extensions via **packages** or **modules** without any need to modify the core of the Mosel language

Mosel modules



Mosel: A modular environment

- **Package** = library written in the Mosel language
 - making parts of Mosel models re-usable
 - deployment of Mosel code whilst protecting your intellectual property
 - similar structure as models (keyword `model` is replaced by `package`), compiled in the same way
 - included with the `uses` or `import` statement
 - definition of new types, subroutines, symbols, control parameters
 - see examples in the [Mosel User Guide](#)

Mosel: A modular environment

- **Module** = dynamic library written in C
 - modules of the Mosel distribution:
 - solver interfaces:
Xpress Optimizer (LP, MIP, QP), Xpress NonLinear, Xpress Kalis (CP)
 - accessing data sources: ODBC, OCI, spreadsheets, XML/JSON, S3
 - system commands; model handling; graphics; HTTP
 - interfaces to external software: R, Matlab, Java, Python
 - write your own modules for
 - connecting to external software
 - time-critical tasks
 - defining new types, subroutines, operators, I/O drivers, control parameters, symbols

Some highlights of module features

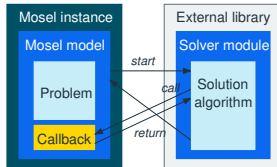
- Interaction with external programs during their execution (callback functions)
- Access to other solvers and solving paradigms (NLP, CP)
- Executing code written with other languages (Java, R, Python)
- Implementation of graphical applications
(*mmsvg*, Xpress Insight)



Module features: solvers

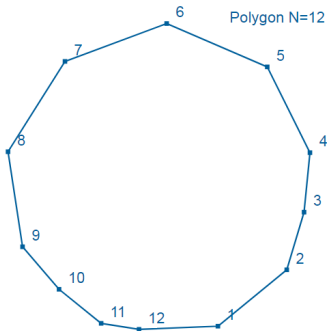
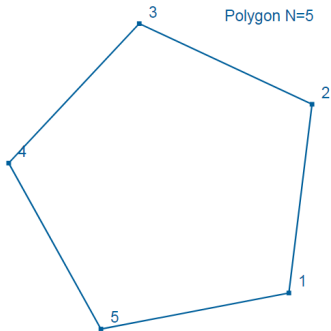
Module mmxprs: Using callback functions

```
uses "mmxprs"  
  
declarations  
  x: array(1..10) of mpvar  
end-declarations  
  
procedure printsol  
  writeln("Solution: ", getsol(Objective))  
  forall(i in 1..10) writeln("x(", i, ")=", getsol(x(i)))  
end-procedure  
  
setcallback(XPRS_CB_INTSOL, ->printsol)
```



Module mmxnlp: Solving an NLP

- What is the greatest area of a polygon of N sides and a diameter of 1?



- Try it yourself:** open `examples/mosel/Nonlinear/polygon1_graph.mos` in your Xpress installation

Module mmxnlp: Solving an NLP

```
model "Polygon"
  uses "mmxnlp"

declarations
  N=5
  RN=1..N
  Area: nlctr
  rho, theta: array(RN) of mpvar
  D: array(RN,RN) of nlctr
end-declarations

forall(i in 1..N-1) do      ! Initialization of NLP variables
  rho(i) >= 0.1; rho(i) <= 1
  setinitval(rho(i), 4*i*(N + 1 - i)/((N+1)^2))
  setinitval(theta(i), M_PI*i/N)
end-do
```

Module mmxnlp:

Solving an NLP

```
forall(i in 1..N-2, j in i+1..N-1) ! Third side of all triangles
    D(i,j) := rho(i)^2 + rho(j)^2 -
        rho(i)*rho(j)*2*cos(theta(j)-theta(i)) <= 1

                                ! Vertices in increasing order
forall(i in 2..N-1) theta(i) >= theta(i-1) +.01

theta(N-1) <= M_PI                ! Boundary conditions

Area:=                            ! Objective: sum of areas
    (sum(i in 2..N-1) (rho(i)*rho(i-1)*sin(theta(i)-theta(i-1))))*0.5
setparam("xnlp_solver", 0)        ! Select the SLP solver
maximize(Area)                    ! Solve the problem

writeln("Area = ", getobjval)
end-model
```

Module kalis:

Constraint Programming

- Example: jobshop scheduling
 - schedule the production of a set of jobs on a set of machines. Every job is produced by a sequence of tasks, each of these tasks is processed on a different machine. A machine processes at most one job at a time.
- Implementation with high-level modeling objects (tasks and resources)

Module kalis:

Constraint Programming

```
model "Job Shop"
  uses "kalis"

  declarations
    JOBS = 1..NJ           ! Set of jobs
    MACH = 1..NM           ! Set of resources
    RES: array(JOBS,MACH) of integer ! Resource use of tasks
    DUR: array(JOBS,MACH) of integer ! Durations of tasks

    res: array(MACH) of cpresource ! Resources
    task: array(JOBS,MACH) of cptask ! Tasks
  end-declarations

  ...                     ! Initialize the data

  HORIZON:= sum(j in JOBS, m in MACH) DUR(j,m)
```

Module kalis:

Constraint Programming

```
forall(j in JOBS) getend(task(j,NM)) <= HORIZON

! Setting up the resources (capacity 1)
forall(m in MACH)
    set_resource_attributes(res(m), KALIS_UNARY_RESOURCE, 1)

! Setting up the tasks (durations, resource used)
forall(j in JOBS, m in MACH)
    set_task_attributes(task(j,m), DUR(j,m), res(RES(j,m)))

! Precedence constraints between the tasks of every job
forall (j in JOBS, m in 1..NM-1)
    setsuccessors(task(j,m), {task(j,m+1)})

! Solve the problem & print solution
if cp_schedule(getmakespan)<>0 then
    writeln("Total completion time: ", getsol(getmakespan))
end-if
end-model
```

Module nlsolv:

Using alternative LP/MIP/NLP solvers

- **Functionality of *nlsolv***
 - export a problem in the .nl file format
 - solve it using an external solver supporting this format
 - no interaction during solving
 - retrieve solution values back into the Mosel model

Module nlsolv:

Using alternative LP/MIP/NLP solvers

- **Functionality of *nlsolv***
 - export a problem in the .nl file format
 - solve it using an external solver supporting this format
 - no interaction during solving
 - retrieve solution values back into the Mosel model
- *nlsolv* does **not** include any solver
 - separate download/installation for every solver
 - for example: <http://ampl.com/products/solvers/open-source>

Module nlsolv:

Using alternative LP/MIP/NLP solvers

- **Functionality of *nlsolv***
 - export a problem in the .nl file format
 - solve it using an external solver supporting this format
 - no interaction during solving
 - retrieve solution values back into the Mosel model
- *nlsolv* does **not** include any solver
 - separate download/installation for every solver
 - for example: <http://ampl.com/products/solvers/open-source>
 - solver choice is configured via parameters
 - NL_SOLVER always needs to be set
 - use NL_SOLVERPATH if executable is not on the path, or executable name is different from solver prefix in NL_SOLVER, or executable requires additional options (define a batch script)
 - optional: specific solver settings in NL_OPTIONS

```
setparam("NL_SOLVER", "xpress")  
setparam("NL_SOLVERPATH", "amplxpress.exe")
```


Project work [K-1]: Alternative solvers

- Inspect the examples of alternative solver types that are provided with the Xpress release:
 - NLP: open and run model `examples/mosel/Nonlinear/moonshot_graph.mos`
 - RO: open and run model `examples/mosel/Robust/roadworks_graph.mos`
 - CP: open and run model `examples/solver/kalis/UG/freqasgn_graph.mos`



Interfacing with analytics tools and other languages

Interfacing with analytics tools and other languages

- Traditional embedding:
 - a host program launches the execution of a Mosel model
 - data exchange in memory
 - Mosel library APIs: C/C++, Java, C#, Python, VBA; Matlab

```
import com.dashoptimization.*;

public class folioobj
{
    public static void main(String[] args) throws Exception
    {
        XPRM mosel;
        XPRMModel model;

        mosel = new XPRM();                // Initialize Mosel
        mosel.compile("foliodata.mos");    // Compile the model
        model = mosel.loadModel("foliodata.bim"); // Load compiled model
        model.run();                      // Run the model
        if(model.getProblemStatus()==XPRMModel.PB_OPTIMAL) // Access solution info
            System.out.println("Objective value: " + model.getObjectiveValue());
    }
}
```

Interfacing with analytics tools and other languages

- New trend:
 - use other languages from within a Mosel model
 - data exchange in memory
 - Mosel modules: R, Java, Python3; Mosel NI: C/C++

Interfacing with analytics tools and other languages

- **R**: pass data to R, execute R functions, retrieve data from R

uses "r"

```
declarations
SHARES = 1..10                                ! Set of shares
DATES: set of string                            ! Historical dates
VAR: array(SHARES,SHARES) of real              ! Variance/covariance matrix of estimated returns
OPEN: array(SHARES,DATES) of real              ! Historical share value at market opening
CLOSE: array(SHARES,DATES) of real             ! Historical share value at market closing
end-declarations
! ... Initialize data ...

! **** Perform some statistics using R ****
Rset('open',OPEN)                             ! Copy array to R environment
Rset('close',CLOSE)
writeln("Covariances at market openings:")
Rprint('cov(t(open))')                        ! Print covariance of share value at market openings
! Calculate and retrieve covariance of mean value in Mosel array VAR
Rgetarr('cov(t((open+close)/2))', VAR)
```

Interfacing with analytics tools and other languages

- **Python:** pass data to Python, execute Python scripts, retrieve data from Python

```
uses "python3"

! **** Perform some statistics using Python ****
! Copy data to Python environment
initialisations to PY_IO_GLOBAL_VAR
  SHARES as 'shares'
  DATES as 'dates'
  OPEN as 'open'
  CLOSE as 'close'
end-initialisations

! Import functions from Python scripts
pyrun('mosel_numpy.py')
pyrun('folioqp_py.py')

! Calculate and retrieve covariance of mean value
pyexec('var = covariance_of_mean(open, close, shares, dates)')
initialisations from PY_IO_GLOBAL_VAR
  VAR as 'var'
end-initialisations
```

Interfacing with analytics tools and other languages

- **Java:** pass data to Java, execute Java code, retrieve result

```
uses "mosjvm"  
  
! **** Perform some statistics in Java ****  
  
! Abort model if we encounter a Java exception  
setparam('jvmabortonexception', true)  
  
! Tell Java to look for classes in working directory  
setparam('jvmclasspath', getparam('workdir'))  
  
! Call Java code to calculate the variance  
forall(s1,s2 in SHARES)  
  VAR(s1,s2) := jvmcallreal('CalcCov.calcValue',  
                           array(t in DR) OPEN(s1,DTIDX(t)),  
                           array(t in DR) OPEN(s2,DTIDX(t)))
```



Package mmsvg: Drawing user
graphs

Package mmsvg:

Drawing user graphs

```
model "Schedule"
  uses "mmsvg", "mmsystem"

declarations
  MACHINES=6; JOBS=6
  job: array(1..JOBS) of integer
  curmachine, curjobs, n1, n2, n3: integer
end-declarations

! Define graph colours
forall(j in JOBS) do
  job(j) := "J"+j
  svgaddgroup(job(j), "Job "+j)
  svgsetstyle(SVG_STROKE, SVG_GRAY)
  svgsetstyle(SVG_FILL, SVG_CURRENT)
end-do

svgsetgraphscale(10)           ! Scale graph size
svgsetgraphlabels("Time", "Machines") ! Set axes labels
```

Package mmsvg:

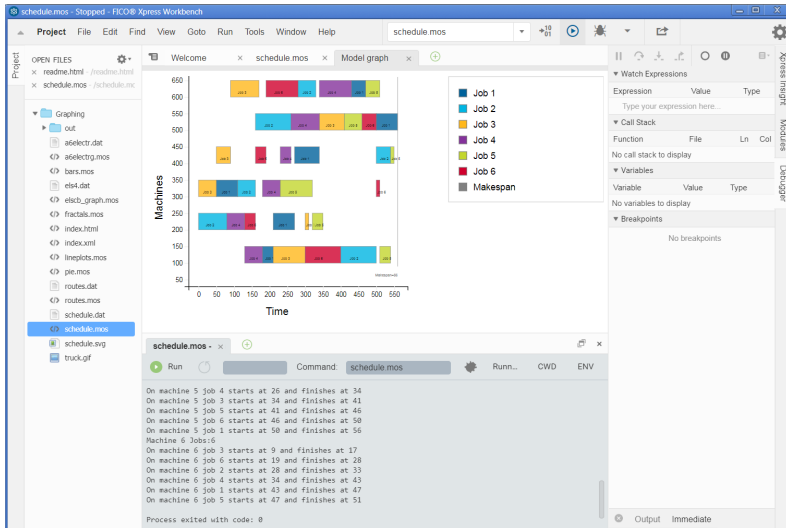
Drawing user graphs

```
fopen("schedule.dat", F_INPUT)
forall (i in 1..MACHINES) do
  readln(n1, n2)                                ! Read machine no. & no. of jobs
  writeln("Machine ", n1, " Jobs:", n2)
  curmachine:= n; curjobs:= n2
  forall(j in 1..curjobs) do
    readln(n1, n2, n3)                          ! Read job no., start & finish times
    writeln("On machine ", curmachine, " job ", n1,
            " starts at ", n2, " and finishes at ", n3)
    svgaddrectangle(job(n1),n2,curmachine*10,n3-n2,0.5*10)
    svgaddtext(job(n1), (3*n2+n3)/4,curmachine*10+1,"Job "+n1)
    svgsetstyle(svggetlastobj,SVG_COLOR,SVG_BLACK)
  end-do
end-do
fclose(F_INPUT)

svgsave("gantt.svg")                            ! Optional: save graphic to file
svgrefresh                                       ! Display the graph...
svgwaitclose                                    ! ...until window is closed
end-model
```

Package mmsvg:

Drawing user graphs



Project work [K-2]: SVG graphic

- Add a pie chart into the model `chess5i.mos`
 - Hint: take a look at the example implementation provided in the file `pie.mos` under the subdirectory `examples/mosel/Graphing` of your Xpress installation
 - Can you include the graphic into the documentation generated with *mosel/doc*?
- Inspect the examples of SVG graph drawing under `examples/mosel/Graphing`, in particular the example in file `elscb_graph.mos` that displays progress graphs of the solving process.



Other functionality of Mosel components

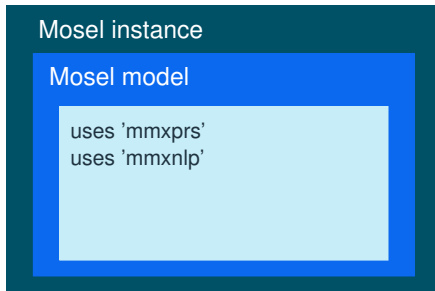
And also

- Working with several models in parallel, possibly in a heterogeneous distributed architecture (module *mmjobs*)
 - see whitepaper [Multiple models and parallel solving with Mosel](#)
- Combining different solvers
 - see whitepaper [Hybrid MIP/CP solving with Xpress Optimizer and Xpress Kalis](#)

Functionality for decomposition and concurrent solving

The "multis":

– multi-solver

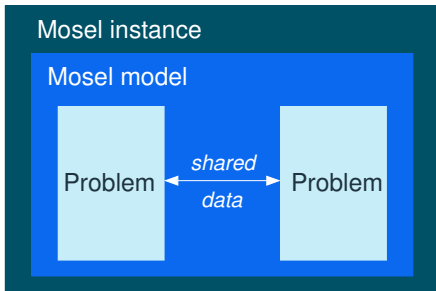


- Choose among the Xpress solvers the one that is best suited for your problem type, or use [several solvers in combination](#) within a single model

Functionality for decomposition and concurrent solving

The "multis":

- multi-solver
- multi-problem

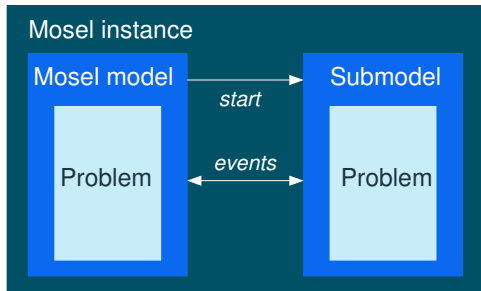


- **Multiple optimization problems** (type `mpproblem`) can be defined within a single optimization model, such problems can share data, and make use of common decision variables

Functionality for decomposition and concurrent solving

The "multis":

- multi-solver
- multi-problem
- multi-model

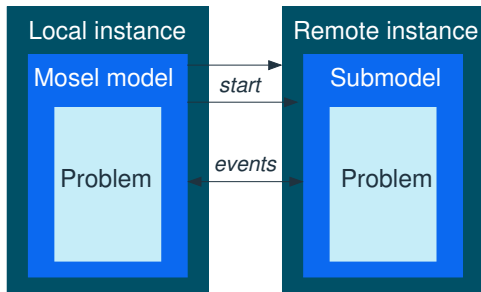


- Multiple optimization problems implemented as [separate model \(files\)](#) make parallel and multithreaded optimization easily accessible

Functionality for decomposition and concurrent solving

The "multis":

- multi-solver
- multi-problem
- multi-model
- multi-node

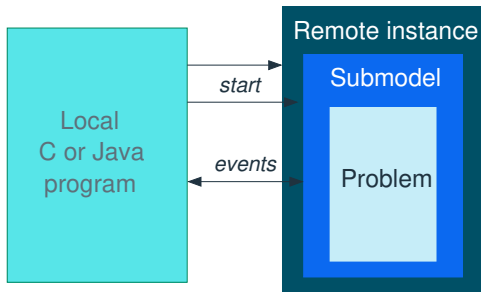


- Extension of multiple model handling to distributed computing using [several Mosel instances](#) opens new perspectives for the implementation of decomposition approaches

Functionality for decomposition and concurrent solving

The "multis":

- multi-solver
- multi-problem
- multi-model
- multi-node



- Remote execution of a Mosel model **without local installation**

Reference material

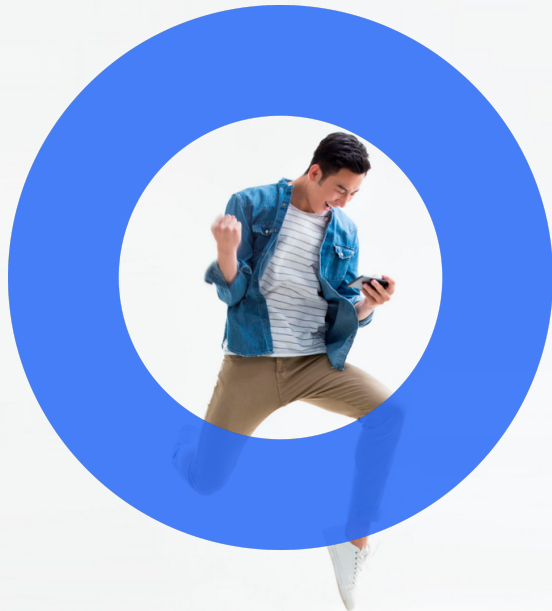
- The modules of the Mosel distribution are documented in the [Mosel Language Reference Manual](#) (with separate manuals for modules *mminsight*, *kalis*, *matlab*, and *nlsolv*).
- The [Mosel Native Interface User Guide](#) explains how to write your own modules.
 - see public Mosel open source repository on Github:
<https://github.com/fico-xpress/mosel>
 - MIP+NLP solver interface templates; *random*, *math*, *moseltesting*, *jobqueue*, *minisat*

Reference material

- The modules of the Mosel distribution are documented in the [Mosel Language Reference Manual](#) (with separate manuals for modules *mminsight*, *kalis*, *matlab*, and *nlsolv*).
- The [Mosel Native Interface User Guide](#) explains how to write your own modules.
 - see public Mosel open source repository on Github:
<https://github.com/fico-xpress/mosel>
 - MIP+NLP solver interface templates; *random*, *math*, *moseltesting*, *jobqueue*, *minisat*
- On the topic of alternative solvers see the whitepaper [Xpress Mosel solver interfaces](#)

Reference material

- The modules of the Mosel distribution are documented in the [Mosel Language Reference Manual](#) (with separate manuals for modules *mminsight*, *kalis*, *matlab*, and *n/solv*).
- The [Mosel Native Interface User Guide](#) explains how to write your own modules.
 - see public Mosel open source repository on Github:
<https://github.com/fico-xpress/mosel>
 - MIP+NLP solver interface templates; *random*, *math*, *moseltesting*, *jobqueue*, *minisat*
- On the topic of alternative solvers see the whitepaper [Xpress Mosel solver interfaces](#)
- The whitepaper [Multiple models and parallel solving with Mosel](#) discusses examples of parallel and distributed computing.





Embedding Mosel models





Deploying Mosel models

Embedding models in applications

- Application design considerations
- Deployment options for Mosel models
- Standalone executable
- The Mosel API

Application design considerations

- **Model development:** once deployed, models often remain in use for periods >10 years $\Rightarrow$ code needs to be suitable for easy maintenance by changing owners
 - conventions in [Mosel model development guidelines for optimization projects](#) document:
 - model code structure and formatting
 - versioning
 - naming standards
 - model debugging
 - error handling
 - documentation

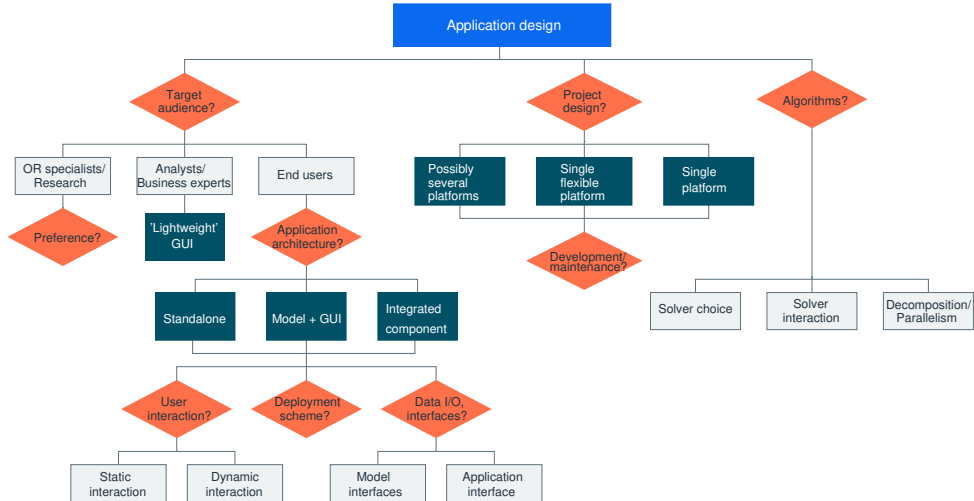
Application design considerations

- **Model development:** once deployed, models often remain in use for periods >10 years $\Rightarrow$ code needs to be suitable for easy maintenance by changing owners
 - conventions in [Mosel model development guidelines for optimization projects](#) document:
 - model code structure and formatting
 - versioning
 - naming standards
 - model debugging
 - error handling
 - documentation
- Comments are essential!

Application design considerations

- **Model formulation:**
 - Solving time varies considerably among problem instances
 - consider trade-offs between decision type and performance required
 - Do not overmodel!
 - overmodeling can be expensive and useless, depending on use-case
 - Consider end-user involvement from the beginning
 - what is the level/type of interaction required for the end-user?
 - Prefer lightweight / modular integration of model
 - be prepared for future changes and upgrades

Application design considerations



Deployment options for Mosel models

- Distribution as BIM file
- Standalone executable
- Embedding in an application via the Mosel API
 - available for C/C++, Java, .NET and VBA
- Xpress Insight application

Reminder: Compiling Mosel models

- Mosel command line:

```
mosel comp mymodel.mos  
mosel comp -g mymodel.mos -o mybim.bim
```

- Mosel language (*mmjobs*):

```
compile("mymodel.mos")  
compile("g", "mymodel.mos", "mybim.bim")
```

- Mosel libraries (e.g. Java):

```
XPRM mosel; XPRMmodel model;  
model = mosel.compile("mymodel.mos");  
model = mosel.compile("g", "mymodel.mos", "mybim.bim");
```

Reminder: Compiling Mosel models

- Workbench:
 - select menu *Run* $\gg$ *Build*

Standalone executable

- *deploy* module: generates an executable containing the BIM file
 - on all supported platforms (generating executables requires C compiler)
 - retrieves arguments from the command line
 - can include additional files (default application data, public keys, certificates)
- Usage (requires C compiler):

```
mosel comp mymodel.mos -o deploy.exe:runmymodel
```

- Output source file `runmymodel.c`:

```
mosel comp mymodel.mos -o deploy.csrc:runmymodel
```

Standalone executable

- *mosdeploy.mos* tool (see XPRESSDIR/examples/mosel/Programming):
 - exploiting *deploy* functionality to generate executables without need for a C compiler by embedding the BIM file into a shell script

Standalone executable

- *mosdeploy.mos* tool (see XPRESSDIR/examples/mosel/Programming):
 - exploiting *deploy* functionality to generate executables without need for a C compiler by embedding the BIM file into a shell script
 - usage examples:

```
mosel mosdeploy -- myfile.mos           creates .bat or .sh file (system-dependent)
mosel mosdeploy -- -cmd myfile.mos       creates .cmd file
mosel mosdeploy -- -o runfile myfile.mos use output name 'runfile'
```



```
mosel mosdeploy -- -exe mosdeploy.mos    creates executable (requires C compiler)
Same as:  mosel comp mosdeploy.mos -o deploy.exe:mosdeploy
mosdeploy myfile.mos                     creates .bat or .sh file
```

Standalone executable

- Retrieving parameter values from the command line
 - replace the parameters block:

```
parameters
  LIMIT=2000
end-parameters
```

by the retrieval of argument value(s):

```
declarations
  LIMIT: integer
end-declarations
```

```
if argc>1 then    ! 1: name of program, all following are arguments
  LIMIT:=integer(argv(2))
else
  LIMIT:=200      ! Default value
end-if
```

Standalone executable

- Usage:

- Compilation (requires C compiler):

```
mosel comp primedeploy.mos -o deploy.exe:runprime
```

- Execution:

- Windows:

```
runprime.exe 500
```

- Unix:

```
runprime 500
```

Standalone executable

- Usage:

- Compilation (requires C compiler):

```
mosel comp primedeploy.mos -o deploy.exe:runprime
```

- Execution:

- Windows:

```
runprime.exe 500
```

- Unix:

```
runprime 500
```

- Compilation using *mosdeploy* tool:

```
mosel mosdeploy.mos -- -o runprime primedeploy.mos
```

- Execution:

```
runprime 500
```

Standalone executable

- Usage:

- Compilation (requires C compiler):

```
mosel comp primedeploy.mos -o deploy.exe:runprime
```

- Execution:

- Windows:

```
runprime.exe 500
```

- Unix:

```
runprime 500
```

- Compilation using *mosdeploy* tool:

```
mosel mosdeploy.mos -- -o runprime primedeploy.mos
```

- Execution:

```
runprime 500
```

- The model can also be run directly as a BIM file:

```
mosel primedeploy -- 500
```

Project work [P-10]: Standalone executable

- Generate an executable for a Mosel model of your choice:
 - transform any runtime parameters as to read their values from the command line
 - if you do not have any C compiler installed:
 - generate the C source instead of the executable, or
 - work with the *mosdeploy* tool (copy the file XPRESSDIR/examples/mosel/Programming/mosdeploy.mos into your working directory)

What is the Mosel API?

- The Mosel language allows you to formulate optimization problems, and develop optimization methods (*i.e.*, use the Optimizer to solve them), as a Mosel model
- The Mosel API (also [Mosel libraries](#)) allows you to embed Mosel models in an application

Programming environments

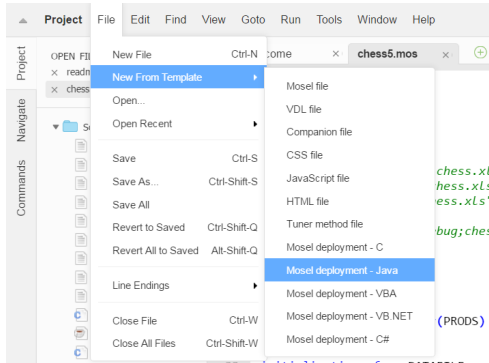
- The Mosel API is available for C/C++, Java, .NET, Python and VBA
- We use Java in the slides, but the functionality applies to all languages, and similar applications can be developed in other languages

Mosel libraries

- Model Compiler Library
 - compiles to a virtual machine
 - binary format architecture independent
- Runtime Library
 - load and run binary (models)
 - access to Mosel internal database (data, solution values, ...)

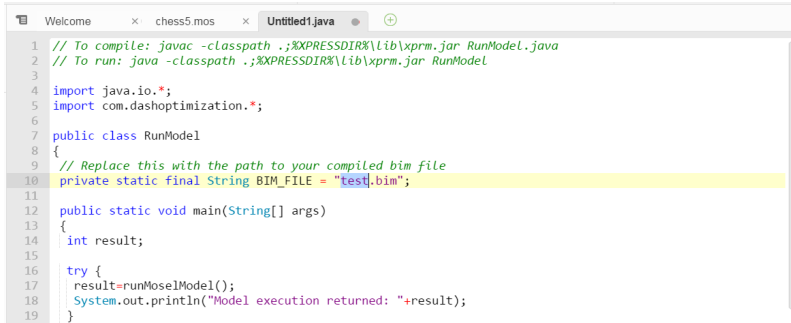
Generating a deployment template

- With Xpress Workbench:
 - compile your model in Xpress Workbench
 - open the menu *File* » *New From Template* then choose the appropriate Mosel deployment file template



Generating a deployment template

- Find the constant with the value `test.bim` near the top of the file and change its value to the name of your BIM file



```
1 // To compile: javac -classpath .;%XPRESSDIR%\lib\xprm.jar RunModel.java
2 // To run: java -classpath .;%XPRESSDIR%\lib\xprm.jar RunModel
3
4 import java.io.*;
5 import com.dashoptimization.*;
6
7 public class RunModel
8 {
9     // Replace this with the path to your compiled bim file
10    private static final String BIM_FILE = "test.bim";
11
12    public static void main(String[] args)
13    {
14        int result;
15
16        try {
17            result=runMoselModel();
18            System.out.println("Model execution returned: "+result);
19        }
```

Generating a deployment template

- Save the file then copy this source file and your BIM file into your project in Microsoft Visual Studio, Eclipse IDE or other development environment
 - Xpress Workbench cannot compile or run source files written in C, C#, Java, VB.NET or VBA; to compile and run the generated source file, use an IDE tool which supports the corresponding language

Mosel library functions

- General:

`XPRM()`, `XPRM.getVersion`, `XPRM.license`, ...

- Model handling:

`XPRM.compile`, `XPRM.loadModel`, `XPRMModel.run`, `XPRMmodel.getResult`,
`XPRMModel.getExecStatus`, `XPRMModel.reset`, ...

- Solution information:

`XPRMModel.getObjectiveValue`, `XPRMModel.getProblemStatus`,
`XPRMMPVar.getSolution`, `XPRMLinCtr.getActivity`, ...

Mosel library functions

- Accessing model objects:

`XPRMModel.findIdentifier`

- Arrays:

`XPRMArray.getDimension, XPRMArray.getIndexSets,
XPRMArray.getFirstIndex, XPRMArray.nextIndex, XPRMArray.get, ...`

- Sets:

`XPRMSet.getSize, XPRMSet.getFirstIndex, XPRMSet.isFixed, ...`

- Handling of modules:

`XPRM.findModule, XPRM.setModulesPath, XPRMModule.parameters, ...`

Project work [C-5]: Model deployment

- Use Workbench to generate a Java program that compiles and runs model `chess5.mos`
- Modify the program so that the model execution uses the data file `chess5.dat`.
- Check the problem status and output the objective value.

Extending the example

- Retrieving detailed solution information and model data

```
XPRMModel model;
XPRMSet prods;
XPRMArray profit, ax;
XPRMMPVar x;
int[] idx = new int[1];
double val;

// Retrieve solution values and problem data
prods = (XPRMSet)model.findIdentifier("PRODS");
profit = (XPRMArray)model.findIdentifier("PROFIT");
ax = (XPRMArray)model.findIdentifier("x");
// Get the first entry of array 'ax'
// (we know that the array is dense and has a single dimension)
idx = ax.getFirstIndex();
do {
    x = ax.get(idx).asMPVar(); // Get a variable from 'ax'
    val = profit.getAsReal(idx); // Get the corresponding value
    System.out.println(prods.get(idx[0]) + ": " + x.getSolution() +
        "\t (profit: " + val + ")"); // Print the solution value
} while(ax.nextIndex(idx)); // Get the next index
```

Extending the example

- Data exchange in memory with host application

```
public class chessio
{
    static int NP = 4;                // Input data
    static final double[] dur        = {3, 2, 2, 3};
    static final double[] wood       = {1, 2, 3, 6};
    static final double[] profit     = {5, 12, 20, 40};
                                     // Array for solution values
    static double[] solution = new double[NP];

    public static void main(String[] args) throws Exception
    {
        int result;
        XPRMModel model;
        XPRM xprm;
```

Extending the example

```
xprm = new XPRM();           // Initialize Mosel
xprm.compile("chess5ioj.mos"); // Compile + load model
model = xprm.loadModel("chess5ioj.bim");
xprm.bind("DUR", dur);        // Associate Java objects with
xprm.bind("WOOD", wood);      // names in Mosel
xprm.bind("PROFIT", profit);
xprm.bind("xsol", solution);
model.execParams = "NP="+NP; // Set runtime parameters
model.run();                  // Run the model
if (model.getProblemStatus()==model.PB_OPTIMAL)
{
    // Check problem status and display the solution
    System.out.println("Objective: " + model.getObjectiveValue());
    for(int i=0;i<NP;i++)
        System.out.println("x(" + (i+1) + "): " + solution[i] +
                             "\t (profit: " + profit[i] + ")");
}
model.reset();
}
```

Deploying to Excel

- Generate VBA code via the Workbench deployment template
 - menu *File* » *New From Template*, select *Mosel deployment - VBA*
 - find the constant with the value `test.bim` near the top of the file and change its value to the name of your BIM file
 - copy the displayed VBA code into the clipboard
- Generate the BIM file
- Create a macro-enabled spreadsheet (.xlsm)

Deploying to Excel

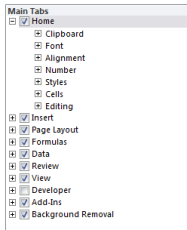
- Select Excel menu *Developer* >> *Insert*
 - select the button object and position it using the mouse
 - assign a new macro to the button and paste the VBA code from clipboard into the macro
- Using Excel menu *Developer* >> *Visual Basic* >> *File* >> *Import File...* add `xprm.bas` from the `include` subdirectory of Xpress to the project
- Select button with right mouse key to edit its text

Deploying to Excel

- If you do not see the *Developer* menu:



- Excel 2007: click on the round button top left then select *Excel Options*; under the *Popular* section, check the option *Show Developer tab*
- Excel 2010 and newer: on the *File* tab, choose *Options*, then choose *Customize Ribbon* and in the list of *Main Tabs*, select the *Developer* check box:



Output redirection to Excel

- Output redirection to file:
 - Edit the macro (after XPRMinit):

```
Call XPRMsetdefstream(0, XPRM_F_ERROR, ActiveWorkbook.path & "\err.txt")
```

```
Call XPRMsetdefstream(0, XPRM_F_WRITE, ActiveWorkbook.path & "\log.txt")
```

- Output redirection to spreadsheet:
 - Edit the macro:

```
nReturn = XPRMsetdefstream(0, XPRM_F_WRITE, XPRM_IO_CB(AddressOf OutputCB))
```


Output redirection to Excel

- Overwriting the same line
 - use Long or LongPtr depending on VBA version (LongPtr for recent/64bit)

```
Public Function OutputCB(ByVal model As LongPtr, ByVal info As LongPtr,  
                        ByVal msg As String, ByVal size As Long) As Long  
    ' Process windows messages so excel gui responds  
    DoEvents  
  
    ' Strip the extra newline character and print to sheet  
    Worksheets("Sheet1").Cells(1, 2) = Mid(msg, 1, Len(msg) - 1)  
End Function
```

Output redirection to Excel

- Multi-line output:

```
Public Function OutputCB(ByVal model As LongPtr, ByVal info As LongPtr,  
                        ByVal msg As String, ByVal size As Long) As Long  
    ' Process windows messages so excel gui responds  
    DoEvents  
  
    ' Strip the extra newline character and print to sheet  
    Worksheets("Sheet1").Cells(ROWNUM, 2) = Mid(msg, 1, Len(msg) - 1)  
    Debug.Print Mid(msg, 1, Len(msg) - 1)  
    ROWNUM = ROWNUM + 1  
End Function
```

- Declare and initialize counter ROWNUM:

```
Global ROWNUM As Integer  
ROWNUM = 1
```

Output redirection to Excel

- Use Mosel's standard functionality for data input and output from/to Excel
 - create named cell ranges in the spreadsheet and use `initializations to/initializations from` in the Mosel model

Summary

- Mosel libraries allow you to embed model programs directly in your application
- Access the solution directly in your application, as alternative to using ODBC
- Enjoy benefits of structured modeling language and rapid deployment when building applications

Summary

- May choose to work with compiled models rather than model source files – provides protection against the user viewing / changing the model
- Compiled models are platform independent

Reference material

- You will find it helpful to refer to the [Mosel Libraries Reference Manual](#)
- The part 'Working with the Mosel libraries' of the [Mosel User Guide](#) documents examples for different programming language interfaces



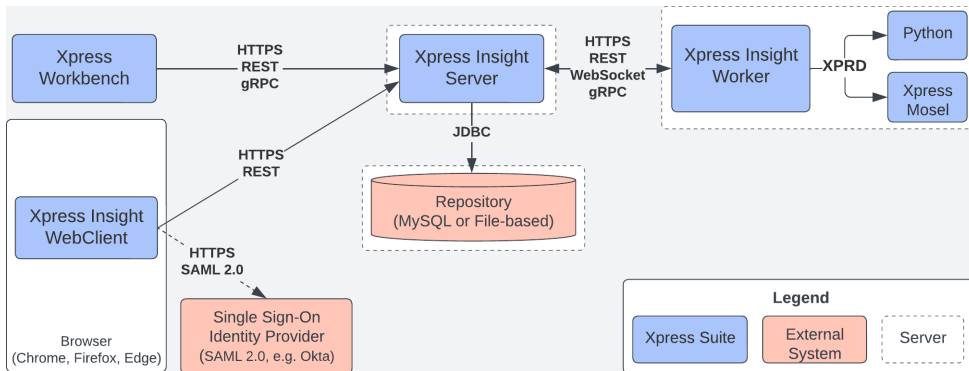
Application development with Xpress Insight

Xpress Insight

- What is Xpress Insight?
 - Xpress Insight is an extensible application for deploying optimization models
 - client-server architecture supporting different types of clients
 - features include
 - data management (files, attachments, scenarios)
 - model execution
 - scenario analysis
 - drag & drop UI creation
 - visualization with complex business workflows
 - security & access management (SAML2)
 - user and role management (authorities)
 - collaborative platform

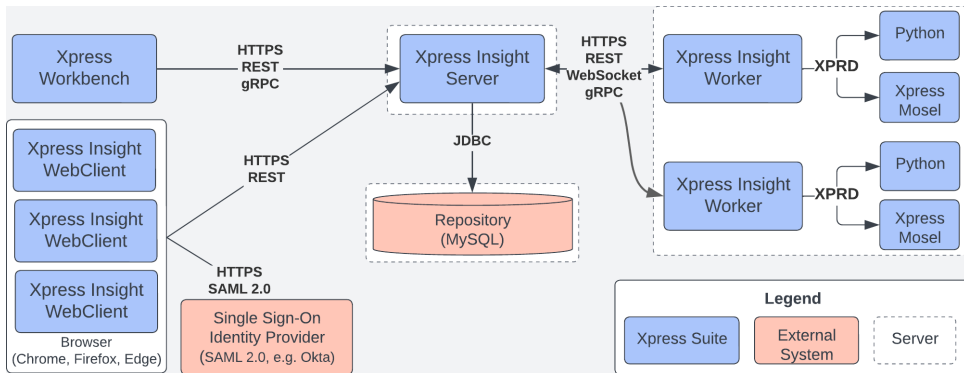
Xpress Insight

- Desktop installation



Xpress Insight

- Multi-user



Xpress Insight: typical workflow

- **Load scenario:** read input data provided in app archive

Xpress Insight: typical workflow

- **Load scenario:** read input data provided in app archive
- **Modify data:** User can change the inputs through the GUI

Xpress Insight: typical workflow

- **Load scenario:** read input data provided in app archive
- **Modify data:** User can change the inputs through the GUI
- **Run scenario:** Execute the Mosel model with current data

Xpress Insight: typical workflow

- **Load scenario:** read input data provided in app archive
- **Modify data:** User can change the inputs through the GUI
- **Run scenario:** Execute the Mosel model with current data
- **Analyze the solution**

Preparing the model file

```
uses "mmxprs", "mminsight"  
parameters  
  DATAFILE='chess5ins.dat'  
end-parameters
```

```
!@insight.manage=input
```

```
public declarations
```

```
  PRODS: range
```

```
! Index set for products
```

```
  RESOURCES: set of string
```

```
! Index set for resources
```

```
  PRODNAME,PROFIT: array(PRODS) of string
```

```
! Product data
```

```
  RESUSE: array(RESOURCES, PRODS) of real
```

```
! Resource usage coefficients
```

```
  RESLIM: array(RESOURCES) of real
```

```
! Resource availability
```

```
end-declarations
```

```
!@insight.manage=result
```

```
public declarations
```

```
  produce: array(PRODS) of mpvar
```

```
! Array of decision variables
```

```
  LimCtr: array(RESOURCES) of lincstr
```

```
! Limit on resource usage constr.
```

```
  TotalProfit: lincstr
```

```
! Objective function
```

```
end-declarations
```

Preparing the model file

```
! Handling model execution modes
case insightgetmode of
  INSIGHT_MODE_LOAD, INSIGHT_MODE_NONE: do
    initializations from DATAFILE           ! Read data from file
    RESUSE [PRODNAME,PROFIT] as "PRODDATA" RESLIM
  end-initializations
  if insightgetmode=INSIGHT_MODE_LOAD then exit(0); end-if
end-do
INSIGHT_MODE_RUN:
  insightpopulate
else
  writeln("Unknown run mode")
  exit(1)
end-case

! ... state the problem ...

! Problem solving via Insight
insightmaximize(TotalProfit)
```


Deployment from Workbench

- Click the 'publish to Insight' button  once your model is ready
 - after entering your Insight credentials (default for desktop installation: admin / admin123) follow the link to the app loaded in the Insight Web Client
- Workbench also supports the editing of Insight app configuration files
 - VDL view definition (custom web views for editing data or reporting results)
 - XML companion file (project configuration settings)
- To create an [Insight app template](#) with the expected subdirectory structure select *Create Xpress Insight Project* at startup (or use menu *Project >> New Project*)

Xpress Insight Web Client

Chess 5 Insight

HOMEAPPJOBSADMIN

DEMO USER ▾HELP ▾

Scenario 1 ▾[Click here to select scenarios](#)

MAIN ▾Chess Production

Resources

AVAILABILITY LIMITS

 Available hours (h)

160

 Available wood (kg)

200

RELOAD DATA

RUN OPTIMIZATION

RESOURCE USAGE

RESOURCES ▾	PRODS ▾	RESUSE ▾
MTime	1	3.0
MTime	2	2.0
MTime	3	2.0
MTime	4	3.0
Wood	1	1.0
Wood	2	2.0
Wood	3	3.0
Wood	4	6.0

Production Plan

 1: 0 pieces

 2: 1 pieces

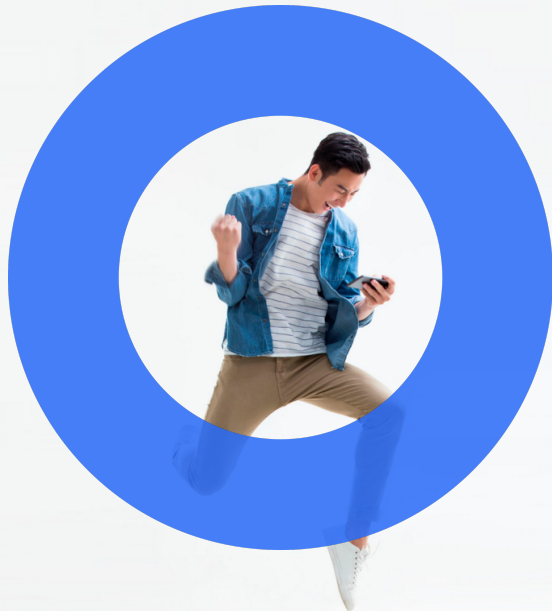
 3: 0 pieces

 4: 33 pieces

 **Total profit: \$1,332.00**

Reference material

- See section *Deployment to Xpress Insight* of the [Getting Started with Xpress](#) manual.
- Xpress Insight documentation: see the [Xpress Insight Web Client User Guide](#) and also the [Xpress Insight Developer Guide](#)





Summary and further information



Summary

- Have seen:
 - FICO Xpress product suite

Summary

- Have seen:
 - FICO Xpress product suite
 - solvers

Summary

- Have seen:
 - FICO Xpress product suite
 - solvers
 - modeling interfaces

Summary

- Have seen:
 - FICO Xpress product suite
 - solvers
 - modeling interfaces
 - development environment

Summary

- Have seen:
 - FICO Xpress product suite
 - solvers
 - modeling interfaces
 - development environment
 - deployment options

Summary

- Have seen:
 - Linear and Mixed Integer Programming (LP and MIP)
 - basic notions of theory
 - examples of standard problem types

Summary

- Have seen:
 - Linear and Mixed Integer Programming (LP and MIP)
 - basic notions of theory
 - examples of standard problem types
 - Working with Mosel
 - formulating optimization problems
 - running and debugging Mosel models

Summary

- Have seen:
 - Linear and Mixed Integer Programming (LP and MIP)
 - basic notions of theory
 - examples of standard problem types
 - Working with Mosel
 - formulating optimization problems
 - running and debugging Mosel models
 - accessing data sources

Summary

- Have seen:
 - Linear and Mixed Integer Programming (LP and MIP)
 - basic notions of theory
 - examples of standard problem types
 - Working with Mosel
 - formulating optimization problems
 - running and debugging Mosel models
 - accessing data sources
 - programming language features

Summary

- Have seen:
 - Linear and Mixed Integer Programming (LP and MIP)
 - basic notions of theory
 - examples of standard problem types
 - Working with Mosel
 - formulating optimization problems
 - running and debugging Mosel models
 - accessing data sources
 - programming language features
 - embedding models in Insight applications for deployment

Summary

- Have seen:
 - Linear and Mixed Integer Programming (LP and MIP)
 - basic notions of theory
 - examples of standard problem types
 - Working with Mosel
 - formulating optimization problems
 - running and debugging Mosel models
 - accessing data sources
 - programming language features
 - embedding models in Insight applications for deployment
- Going further:
 - advanced optimization techniques
 - infeasibility handling, accessing multiple solutions, user-provided solutions, user cuts, Nonlinear Programming, ...

Summary

- Have seen:
 - Linear and Mixed Integer Programming (LP and MIP)
 - basic notions of theory
 - examples of standard problem types
 - Working with Mosel
 - formulating optimization problems
 - running and debugging Mosel models
 - accessing data sources
 - programming language features
 - embedding models in Insight applications for deployment
- Going further:
 - advanced optimization techniques
 - infeasibility handling, accessing multiple solutions, user-provided solutions, user cuts, Nonlinear Programming, ...
 - advanced modelling topics
 - decomposition schemes, parallel and distributed computing
 - tools: Profiler, *mosel*doc, *mosel*test
 - programming language APIs for embedding models or invoking other programs/languages
 - Mosel Native Interface (NI), language extensions (modules and packages)

Further information

- FICO Xpress online resources
 - Online documentation and examples:
<http://www.fico.com/fico-xpress-optimization/docs/latest>
 - Online searchable examples database:
<http://examples.xpress.fico.com/example.pl>
 - Optimization community (online discussion group, video library, blogs):
<http://community.fico.com/optimization>

Further information

- FICO Xpress online resources
 - Online documentation and examples:
<http://www.fico.com/fico-xpress-optimization/docs/latest>
 - Online searchable examples database:
<http://examples.xpress.fico.com/example.pl>
 - Optimization community (online discussion group, video library, blogs):
<http://community.fico.com/optimization>
 - FICO Xpress Academic Partner Program (free)
<http://www.fico.com/app>

Further information

- FICO Xpress online resources
 - Online documentation and examples:
<http://www.fico.com/fico-xpress-optimization/docs/latest>
 - Online searchable examples database:
<http://examples.xpress.fico.com/example.pl>
 - Optimization community (online discussion group, video library, blogs):
<http://community.fico.com/optimization>
 - FICO Xpress Academic Partner Program (free)
<http://www.fico.com/app>
- Public Xpress open source repository
 - Github:
<https://github.com/fico-xpress>
 - Run Xpress notebooks online:
<https://github.com/fico-xpress/python-notebooks>



<http://www.fico.com/xpress>